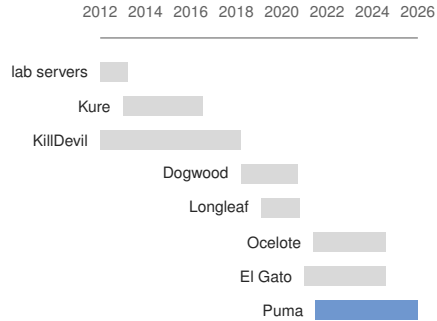
X1. RETIRED CLUSTERS AND WORKFLOWS, 2012–2024

How IBAMR and IB2d runs were done on the lab's own servers, UNC's Kure, KillDevil, Dogwood and Longleaf, and Arizona's Ocelote and El Gato: logins, storage, modules, schedulers and job scripts, each mapped to today's Puma; labelled legacy



Laura A. Miller

Department of Mathematics, University of Arizona

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT THIS ARCHIVE CHAPTER IS FOR

- Reading old notes. Twelve years of course notes, lab notes, job scripts and papers' methods sections refer to machines that no longer run: the lab's own Linux servers, UNC's Kure, KillDevil, Dogwood and Longleaf, and the University of Arizona's Ocelote and El Gato. This chapter keeps how they were used, so that an old note or script can be read and its run understood.
- Legacy. Every historical listing is marked "historical, do not run", and every slide carries the LEGACY label; a dash or curly quote that a shell would not accept is printed red. The host names, queues, modules and folders are gone; several commands carry slips that this chapter points out.
- Most slides point to today's way: chapters 1 (Puma, Slurm, storage), 2 (installing IBAMR), 3 (building and running) and 31 (VisIt on the cluster).

> Today the series runs on one cluster, Puma at the University of Arizona, with Slurm, IBAMR built by autoibamr, and a CMake build (chapters 1–3). Nothing in this chapter is needed to follow the rest of the series.

Building the software on these machines is chapter X2; the code changes from one installed IBAMR to the next are chapter X3.

The sources themselves, compared in October 2026.

WHERE THESE NOTES CAME FROM (1/2): 2012–2016

source	year	machines	author
IBAMR ReadMe deck (lab)	2012	lab servers; KillDevil	the Miller lab (author not recorded)
“How to use the 3D IBAMR created by Boyce Griffith”	2012	a lab server	Shannon Jones (“Tutorial by S. Jones”)
forces install deck	2013	lab servers; KillDevil	the Miller lab (author not verified)
“Install THLinux and IBAMR”; “Running IBAMR on THLinux”; “Running IBAMR on Killdevil”	2011–13	the lab’s servers; KillDevil	the instructor
MATH 564 homework 1	2013	a lab server; KillDevil, Kure	the instructor
“IBAMR Killdevil Module” (March 26, 2014)	2014	KillDevil	the instructor
“Using IBAMR Module on Kure - Instructions for Undergraduates”	2014	Kure, KillDevil	not recorded
“The Joy of Immersed Boundary Methods with Adaptive Mesh Refinement” (notes from the 2014 summer course)	2014	Kure	Dennis Evangelista (author line: “People and Dennis Evangelista”)
“IBAMR Tutorial: UNC Edition” (also MATH 769, Lecture 3, January 2015)	2014–15	Kure	not stated
“Party time in on super computers! (. . . or how to run simulations on Kure/Kill Devil)”	2014	Kure, KillDevil	Nick Battista
MATH 769, Lecture 1 (slide 11)	2015	Kure	the instructor
“Getting Running on IB2D and IBAMR” (MATH 768)	2016	Kure, KillDevil	not stated
IB talk, 7 September 2016	2016	Kure, KillDevil	Nick Battista and Laura A. Miller

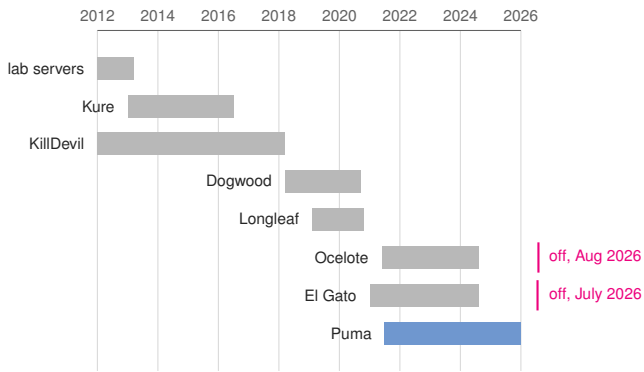
The sources themselves, compared in October 2026. Where a source does not state its author, it stays unattributed.

WHERE THESE NOTES CAME FROM (2/2): 2017–2024

source	year	machines	author
“IBAMR Tutorial: Getting Started on UNC cluster”	2017–18	KillDevil	not stated
TubeHeart “Read me”	2017	KillDevil	the instructor
“1 - Get killdevil account and software”	2018	KillDevil	the instructor
“7-IBAMR- Running many simulations Part 2”	2018	KillDevil	the instructor (two scripts by a student)
“Run your first IBAMR simulation”	2018	KillDevil	Nick Battista (2014), adapted by the instructor
VisIt decks: “Visit from command line and Python”, “Analyzing your IBFE simulation”, “Resampling AMR grids”	2017–18	KillDevil	not stated; parts by Christopher Strickland and Lindsay Waldrop
IBFE “Getting Started” decks	2014–18	KillDevil	adapted from Alex Hoover’s tutorial
“2-Run your IBAMR simulation on Dogwood”; “Debugging Tips” (2019)	2018–20	Dogwood	the instructor’s course decks
Lecture 8: COMSOL workflow, export and run remotely	2019–20	Dogwood, Longleaf	the instructor
“Get dogwood account and run a simulation” (Lecture 1)	2020	Dogwood	the instructor, adapted from Nick Battista (2014)
IBFE lecture 5 (“Plans for June”); Lecture 6	2020; 2025	Dogwood; Puma	the instructor
VisIt on the cluster (Lecture 5 of the data course)	2020	Dogwood, Longleaf	G. Hobson
the course job scripts and Makefiles (`examples/legacy`)	2014–24	KillDevil, Dogwood, Ocelote, Puma	the course
UA cluster decks (“UAClusterIb2dIBAMR”, “UA-HPC”)	2021–22	Puma, Ocelote, El Gato	the instructor
“Get HPC account and run a simulation” (UA Lecture 1); Lecture 0	2024	Puma, Ocelote, El Gato	the instructor (Lecture 1 adapted from Nick Battista, 2014)

The sources themselves, compared in October 2026.

THE MACHINES, 2012–2026: A TIMELINE



- The bars mark the years in which the course's notes used each machine, not the machines' lifetimes. No source in the course's files states when Kure, KillDevil, Dogwood or Longleaf were retired.
- January 2018: "the server used in most of the screenshots is kure. We now only use the server killdevil."
- Ocelote moved to CentOS 7 and Slurm in June–July 2021 (an OnDemand banner: "Ocelote downtime scheduled on 2021-06-01, 08:00 thru 2021-07-01, 08:00 for Upgrade to CentOS 7 and Slurm"; the deck of 11 July 2021: "This cluster should be online July 12").
- El Gato was shut down on 29 July 2026 and Ocelote on 17 August 2026 (chapter 1); Puma is the only UA cluster.

The dates stated in the sources; chapter 1 for the 2026 shutdowns.

THREE WAYS A RUN WAS STARTED, AND TODAY'S

Lab server, 2012: no scheduler 2012 tutorial (fixed) • historical, do not run

```
screen
mpirun -np 4 ./main3d input3d
# Ctrl+A Ctrl+D: detach
ps -u <user>
kill <pid>
```

LSF: Kure, KillDevil, 2012–18 2014 notes • historical, do not run

```
bsub -q week -n 8 mpirun ./
      ↪ main2d input2d
bjobs
bkill <jobid>
```

Slurm: Dogwood 2018–20; Arizona 2021–24 after the 2020 notes; `<onyen>` for the user ID • historical, do not run

```
sbatch script.sh
squeue -u <onyen>
scancel <jobid>
```

Today: Puma (chapters 1 and 3)

```
sbatch job.slurm
squeue --me
scancel <jobid>
```

- On the lab's servers a run was started by hand and shared the machine with everyone else's; the users kept the peace by rules (slide 15).
- With LSF and Slurm a scheduler starts the run on compute nodes when the cores are free. On the UNC clusters the login node was for editing, building and submitting; on Puma it is for editing and submitting only, and builds run on a compute node too (chapters 1 and 3).
- Slurm's commands have barely changed since Dogwood; what changed is the job script around them (chapter 3).

Shannon Jones's 2012 tutorial (with the en dashes fixed); Nick Battista's 2014 notes; the 2020 Dogwood notes; chapter 3.

THE LAB'S OWN LINUX SERVERS (2012)

- In 2012 the lab ran IBAMR on its own Ubuntu servers (Ubuntu 11.04 in the screenshots), shared by everyone in the lab. Their names are withheld here.
- The ReadMe deck began with "Logging into a linux server": download the SSH client that UNC's IT services offered, "Open ssh, and click on Quick Connect", and give the host name and your user name. Its slide also named a Unix tutorial on the web for newcomers.
- IBAMR was built in each user's home folder (`$HOME/sfw`, chapter X2), and the examples ran from its build folder: `$HOME/sfw/ibamr/ibamr-objs-opt/examples/...`.
- By September 2013 the two servers ran THLinux, UNC's "Tar Heel Linux", with one IBAMR built as root in `/usr/bin/sfw` for everyone (slide 13; chapter X2); when they changed from Ubuntu is not stated.

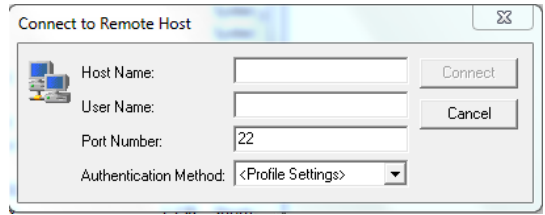
> The first slide of the 2012 ReadMe also gave the server's login details. They are withheld here: login details never belong in shared notes or slides.

Today: you log in to Puma through the university's bastion host with your own NetID and two-step login (chapter 1).

The Miller lab's IBAMR ReadMe deck (2012), slides 1–4; Shannon Jones's 2012 tutorial, slide 3 and screenshots; the instructor's "Running IBAMR on THLinux" (2013), slides 2 and 4–5; host names withheld.

LOGGING IN AND COPYING FILES WITH THE 2012 SSH CLIENT

- Two programs from UNC's IT services, "SSH Secure Shell Client" and "SSH Secure File Transfer" (an ONYEN, UNC's user ID, was needed to download them).
- File transfer: Quick Connect, then the host name and your user name, then the password; the window shows the local files on the left and the server's on the right, and files are dragged across. The 3D IBAMR files were "under `sfw/ibamr/ibamr-objs-opt/examples/samples_wings/`".
- Shell: the same Quick Connect dialog opens a terminal on the server.



The client's "Connect to Remote Host" dialog (2012), empty.

Today: a terminal (Mac, Linux, Windows OpenSSH or WSL, or PuTTY) for the shell, and FileZilla or `scp` for files (chapter 1).

Shannon Jones, "How to use the 3D IBAMR created by Boyce Griffith" (May 2012), slides 3–13; the screenshot from slide 6, cropped to the dialog.

FINDING YOUR WAY ON THE SERVER: CD, LS, TAB, EMACS

2012 tutorial, slides 14–21 • historical, do not run

```
cd sfw            # into a folder
cd ..             # one folder up
ls                # folders are shown in blue
# Tab completes a file or folder name
emacs update_target_point_positions.C   # edit a file; Ctrl
      ↪ +X Ctrl+C to leave
make clean
make main3d
```

- The 2012 tutorial taught the shell from scratch, a command per slide.
- The files a user changed for a new run (slide 20): `input3d`, `update_target_point_positions.C`, and the structure files `.vertex`, `.spring`, `.beam`, `.target`.
- After an edit: `make clean` and `make main3d`, then the run (slide 12).

Today: the shell is chapter 1's subject (with nano and vi in place of emacs); the structure files are chapter 6's.

Shannon Jones, "How to use the 3D IBAMR created by Boyce Griffith" (2012), slides 14–21 (the commands retyped; the comments are this chapter's).

BUILDING AND REBUILDING AN EXAMPLE IN YOUR HOME FOLDER (2012) (1/2)

ReadMe deck, slide 3, “Running the code” • historical, do not run

```
cd $HOME/sfw/ibamr/ibamr-objs-opt/examples/moving_plates2d
IBAMR_SRC_DIR   = /path/to/IBAMR
IBAMR_BUILD_DIR = /path/to/ibamr-objs-directory
% make clean
% make distclean
% make main2d
% ./main2d input2d
```

slides 7, 8 and 10 (the % prompts left out; a slide’s commands joined with ;) • historical, do not run

```
# 3D: in .../examples/moving_plates3d
make clean
make cleandist
make main3d
./main3d input3d
# after editing update_target_point_positions.C:
rm -f main2d; make clean; make cleandist; make main2d
# other examples:
cd IB; cd staggered_grid; cd explicit; make examples; cd ex0;
      ↪ make main2d; ./main2d input2c
```

- The % is the deck’s prompt; the two Makefile lines are the ones each example’s Makefile had to point at the user’s own IBAMR (chapter X3 dates code by them).

Today: CMake in a `build` folder, and runs on compute nodes (chapter 3).

The Miller lab’s IBAMR ReadMe deck (2012), slides 2–3 and 7–10.

BUILDING AND REBUILDING AN EXAMPLE IN YOUR HOME FOLDER (2/2): WHAT THE DECK SAYS

- “Simple clap and fling” (`moving_plates`): the left wing translates left and the right one right; a time step too large shows as “an error quickly” or “very noisy” results, and `dt` is set in the input file.
- The parameters (slide 9): `plate3d_left_512.{vertex,spring,beam,target}`, made by MATLAB scripts run twice, once per wing.
- Slips: `make cleandist` is a slip for `make distclean`, which the same deck writes on slides 3 and 7; `input2c` is `input2d`; the deck calls a generator “generate_meash3d.m” (`generate_mesh3d.m`).

The Miller lab’s IBAMR ReadMe deck (2012), slides 2–3 and 7–10.

RUNNING WITHOUT A SCHEDULER: SCREEN, MPIRUN, PS, KILL

2012 tutorial, slide 27, “Overview of IBAMR Commands” (as written; two lines of its prose left out) • historical, do not run

```
make clean
make main3d
screen (followed by enter)
mpirun –np 4 ./main3d input3d
<Ctrl+A><Ctrl+D> (to exit screen)
ps –u<user>
kill #####
```

ReadMe deck, slide 4, “Doing a parallel run” • historical, do not run

```
mpirun -np 4 ./main2d input2d
```

- `screen` starts a terminal session that survives logging out; Ctrl+A then Ctrl+D detaches from it, and the run goes on; `screen -r` returns to it.
- `mpirun -np 4` starts four processes on the server itself; `ps -u <user>` lists your processes with their IDs, and `kill` stops one.
- Slips: the slides type en dashes (`–np`, `–u`, in red), which the programs (`mpirun`, `ps`) do not read as options; the speaker notes start the command with a capital M (`Mpirun`), and Linux commands are case-sensitive. (`ps -u<user>`, with no space, is fine: the tutorial’s own screenshot shows it working.)
- The tutorial’s `ps` screenshot shows `main3d` processes that had used up to four days of CPU time each on the shared server.

Today: never run on a login node; Slurm runs the job (chapter 3).

Shannon Jones’s 2012 tutorial, slides 22–27 and notes (the lab user name replaced by `<user>`); the Miller lab’s ReadMe deck, slide 4.

THE LAB'S SERVERS IN 2013 (1/2): SCREEN AND THE LIBRARY PATH

"Running IBAMR on THLinux" (2013), slide 4, as written • historical, do not run

```
Go to the folder where you have your example code

(Advanced) On a new system, check the Makefile to make sure that
IBAMR_SRC_DIR   = /path/to/IBAMR
IBAMR_BUILD_DIR = /path/to/ibamr-objs-directory

For the two-dimensional case do
% screen
% export LD_LIBRARY_PATH=/usr/bin/sfw/linux/openmpi/1.4.3/lib
% make clean
% make main2d
% ./main2d input2d &
& Ctrl+A Ctrl+D (this exits the screen and it will run in the
      ↪ background.
```

- By September 2013 the two servers ran THLinux with one IBAMR built as root in `/usr/bin/sfw` (chapter X2); the OpenMPI library path had to be typed in the terminal before each run ("every time you run main2d/3d"); adding it to `.bash_profile` and `.bashrc` "does not seem to fix it".
- Ctrl+A then Ctrl+D *detaches* from `screen` (it does not exit it): the run goes on, and `screen -r` returns to it later, which none of the old notes says. The `&` before "Ctrl+A" is probably the `%` prompt mistyped (the keys are pressed, not typed), and inside `screen` the `&` after the command is not needed.

Today: a batch job on the compute nodes, not `screen` on a shared machine (chapter 3).

The instructor's "Running IBAMR on THLinux" (created September 2013), slides 2, 4 and 6–7; "Install THLinux and IBAMR", slides 2, 5 and 21; "Running IBAMR on Killdevil" (2013), slide 5.

THE LAB'S SERVERS IN 2013 (2/2): A TIGHTER TOLERANCE

slides 6–7, one line from each: "Reducing error tolerance in fluid solver"; "Reduced error tolerance and parallel submission command" • historical, do not run

```
./main2d input2d -stokes_ksp_rtol 1.0e-9
mpirun -np 4 ./main2d input2d -stokes_ksp_rtol 1.0e-9
```

- `-stokes_ksp_rtol 1.0e-9` tightens the Stokes solver's relative tolerance: "Often this is needed for convergence". It was also given after `mpirun` (slide 7) and, on KillDevil, after `bsub`: `bsub -q week -n 16 mpirun ./main2d input2d -stokes_ksp_rtol 1.0e-9` ("Running IBAMR on Killdevil", slide 5).

Today: the same option works; the input file's solver block is the better place for it (chapter 16).

The instructor's "Running IBAMR on THLinux" (created September 2013), slides 2, 4 and 6–7; "Install THLinux and IBAMR", slides 2, 5 and 21; "Running IBAMR on Killdevil" (2013), slide 5.

SHARING ONE SERVER: THE 2012 RULES

From the ReadMe deck, “Doing a parallel run”:

- one server “has 24 processors, and” the other “16”;
- “You want to use a power of 2”;
- “Do not use all of the processors! Use no more than 16” on the first “and 8” on the second;
- use fewer if others are running, and plan big runs together at lab meetings.

> These rules are what a scheduler enforces today. On Puma, Slurm gives each job the cores it asked for, holds the others until cores are free, and stops a job at its time limit (chapters 1 and 3).

Slurm and IBAMR accept any number of processes; the series’ job scripts ask for 4 (chapter 3).

The Miller lab’s IBAMR ReadMe deck (2012), slide 4 (server names withheld).

WATCHING THE TIME STEP IN THE LOG (2012)

ReadMe deck, slide 5, “Output (2D sims)” • historical, do not run

```
% grep "dt = " IB2d.log
% tail -f IB2d.log | grep "dt = "
```

- The 2D examples wrote their visualization files to `viz_IB2d` and their diagnostics to `IB2d.log`.
- `grep` prints the time-step lines; `tail -f` follows the log as the run writes it.

> “If you see that the timestep size decreases dramatically, this usually means that the computation has gone unstable. Reduce the value of ‘DT’ in input2d and restart the model.”

The advice holds today. `IB2d.log` is the log file named in the input (`log_file_name`) of IBAMR’s 2D examples, not a file of the IB2d software.

Today: the job’s log and `.out` file (chapter 3); time steps and stability (chapter 16).

The Miller lab’s IBAMR ReadMe deck (2012), slides 2 and 5.

A 2013 HOMEWORK ON THE LAB SERVER

MATH 564, homework 1 (due 19 February 2013), told students where to run two of its examples (the bristles and the multiple boundaries):

- they “will need to be run on the new force installation of IBAMR, or on” a lab server (the forces build of chapter X2);
- “Please create a folder in” the lab’s shared folder “if you wish to run simulations on” that server;
- “Copy these files to killdevil (kure) or your folder on” the lab server.

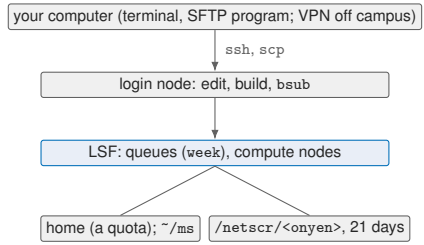
The homework itself (end times, plots at cycles 40000 and 100000, Reynolds-number questions) is in Appendix C, which maps it to today’s examples.

In 2013 a course thus used three kinds of machine at once: the lab’s servers, KillDevil, and Kure. The next part of this chapter is UNC’s two clusters.

The instructor’s MATH 564 homework 1 (Spring 2013); server and folder names withheld.

KURE AND KILLDEVIL AT A GLANCE (2012–2018)

- Two UNC clusters used from 2012 to 2018: Kure (“the machine where IBAMR jobs are run”, the 2014 notes say) and KillDevil. They shared the same software tree (`/nas02/apps`) and modules, so the same Makefile worked on both.
- The scheduler was LSF: `bsub` submits, `bjobs` lists, `bkill` stops (slides 32–33); the notes name the queues `week` (“longest queue”), `day`, `int`, `debug` and `ms` (copies to mass storage).
- Software by modules: `module list`, `module add`, `module initadd`; IBAMR itself from 2014 (`ibamr/3803`, then `ibamr/2016-11`).
- Storage: a home folder with a quota, the scratch space `/netscr/<onyen>`, emptied of files 21 days after they were last changed, and mass storage (`~/ms`); Kure and KillDevil shared all three.
- January 2018: “the server used in most of the screenshots is kure. We now only use the server killdevil.”



The 2014 notes (Dennis Evangelista; “IBAMR Tutorial: UNC Edition”; Nick Battista); “Getting Running on IB2D and IBAMR” (2016), slide 9; “Run your first IBAMR simulation” (2018); “Using IBAMR Module on Kure” (May 2014). Retirement dates are not stated in the course’s files.

GETTING AN ACCOUNT: ONYEN SERVICES

- Every UNC person had an ONYEN (“Only Name You’ll Ever Need”), the user ID for every campus service.
- A cluster account was one more service: on the ONYEN web page, “Select subscribe to services, and then select Kure” (2014; later KillDevil, then Dogwood).
- The 2014 summer-course notes put it the same way: Kure access through ONYEN Services, after which Kure is “the machine where IBAMR jobs are run”.
- January 2018: “Subscribe to Services”, then “KillDevil Cluster”. The page could answer “You are ineligible for the following services for the reasons cited: KillDevil Cluster: missing `'(LIVE|EXCH)'` prerequsite.” [*sic*]; an e-mail to Research Computing then did it (slide 58, where the 2020 Dogwood guide keeps this text).

> Today: a University of Arizona HPC account comes from the HPC portal; a faculty member (your PI) sponsors you and adds you to their group, whose allocation your jobs use (chapter 1).

“IBAMR Tutorial: UNC Edition” (May 2014), slide 2; Dennis Evangelista’s notes (2014), section 2; “1 - Get killdevil account and software” (January 2018), step 1; “Get dogwood account and run a simulation” (2020), step 1.

FROM YOUR COMPUTER: A TERMINAL, AN SFTP PROGRAM, AND VPN

- A terminal: on a Mac, the Terminal (found with Spotlight); on Windows, PuTTY or the SSH program from UNC's software site; on Linux, OpenSSH (`ssh`, `sftp`), installed with `apt-get`.
- An SFTP program to move files: FileZilla ("Install filezilla or another sftp program to move files on and off kure", the 2015 first-week checklist).
- VPN off campus: "you will also need to use VPN (very private network)" [*virtual* private network]; on Ubuntu, `vpnc` and `network-manager-vpnc`.
- A plain-text editor that "won't add random hidden characters (like Word)": Notepad++ or Sublime (2018). In FileZilla or Fetch: host `kure.unc.edu`, port 22, remote folder `/netscr/onyen` (2014).
- The 2015 first-week checklist also asked for two screenshots, posted to the course site: "a terminal opened that is logged into kure" and VisIt.

The 2014 terminal basics (Nick Battista's notes):

`pwd`	where am I
`cd ..`, `cd [folder_name]`	up one folder; into a folder
`ls`, `ls -a`	list (with hidden files)
`mkdir [name]`	make a folder
`rm [name]`, `rm -r [folder_name]`	remove a file; a folder
`cp [file1] [file2]`	copy a file
`cp -r [folder_name1] [folder_name2]`	copy a folder

Today: chapter 1 (terminals, FileZilla, no VPN for SSH).

"IBAMR Tutorial: UNC Edition" (2014), slide 2; Dennis Evangelista's notes (2014); Nick Battista, "Party time in on super computers!" (August 2014), pages 1–2; MATH 769 Lecture 1 (January 2015), slide 11; "Using IBAMR Module on Kure" (May 2014); "1 - Get killdevil account and software" (January 2018), steps 2–6.

LOGGING IN, AND THE RULE ABOUT THE LOGIN NODE

2014 notes; the second line from the 2017–18 KillDevil deck, slide 2

(placeholders for the ONYEN) • historical, do not run

```
ssh <onyen>@kure.unc.edu
ssh <onyen>@killdevil.unc.edu
# the ONYEN password, then:
cd /netscr/<onyen>/
```

> “repeat after me – WE DO NOT RUN SIMULATIONS ON THE LOGIN NODE”

- The login node is the machine `ssh` lands on, shared by everyone logged in. It was for editing, building and submitting; runs went to the compute nodes through the scheduler.
- The prompt showed the login node: `kure-login1`, `killdevil-login1`, `killdevil-login2`. The 2014 Kure deck itself shows a `killdevil-login1` prompt in its listings (slides 3 and 5).
- Dennis Evangelista’s notes write `ssh kure.unc.edu` without the ONYEN, which works only when the user name on your own computer is the ONYEN (or an SSH configuration file supplies it).

Today: `ssh <netid>@hpc.arizona.edu`, then `shell`; a stricter rule: neither runs nor builds on the login node (chapter 1).

Nick Battista, “Party time in on super computers!” (2014), pages 2–3 (Part I, steps 7–9); “IBAMR Tutorial: UNC Edition” (2014), slides 2–5; Dennis Evangelista’s notes (2014), section 3.1.

HOME, /NETSCR AND KEEPING DATA

- “It is important to remember NOT to run simulations in your home directory!! All simulations need to be run in the scratch file directory /netscr/onyen/” (2014).
- “the files in scratch file directory will be deleted in 21 days after the modification of the file”; “files in the netscr directories are deleted 21 days after they are last modified”.
- “Note that your home directory only has $\sim$ 5 GB. Also, all files are deleted every few weeks from netscr, so do not store data there long term!” (2016).
- The home quota differed by account and year: “My quota is 10,240 Megabytes, and the hard limit is 15360 Megabytes” (one account, 2014); `quota` showed the use and both limits, `du -hs` a folder’s size; “if you exceed your quota, your simulation can be terminated.”
- Results were copied off “at least within a week” (slide 36), or to mass storage (next slide).

Today on Puma: `/home` (50 GB), `/xdisk/<pi>/<netid>` for runs (150 days, extendable once by 150), `/groups/<pi>` (500 GB) for what the group keeps (chapter 1). The rule is the same: run in the scratch space, keep results elsewhere.

“IBAMR Tutorial: UNC Edition” (2014), slides 3 and 6; Dennis Evangelista’s notes (2014), section 3.2; “Getting Running on IB2D and IBAMR” (2016), slide 9; Nick Battista’s notes (2014); “Using IBAMR Module on Kure” (May 2014), “General commands”.

MASS STORAGE ON KURE AND KILLDEVIL

"Using IBAMR Module on Kure" (May 2014) • historical, do not run

```
cd ~/ms
bsub -q ms cp /netscr/onyen/text.txt /ms/home/o/n/onyen
```

"Running many simulations Part 2" (March 2018), slide 15 • historical, do not run

```
bsub -q ms cp /netscr/myonyen/output/* /ms/home/m/y/myonyen/
      ↪ saved_output
```

`onyen` and `myonyen` are the sources' own placeholders; `o/n` and `m/y` are the first two letters of the user ID.

- The `ms` folder in each home was the door to UNC's mass storage, a tape archive for finished or very large files: not a place to work, and not a backup.
- Jobs running under LSF could not read it; data had to be copied back to `/netscr` first.
- Many small files went in as one `tar` or `zip` file.
- Large copies went through LSF's `ms` queue, on hosts with fast access to the archive, not on the login node.

Today: `/groups/<pi>` for what the group keeps, and your own copies off the cluster (chapter 1).

"Using IBAMR Module on Kure - Instructions for Undergraduates" (May 2014; author not recorded), section "Mass Storage" (paraphrasing UNC's help page); "7-IBAMR-Running many simulations Part 2" (March 2018), slide 15.

WHAT THE `module` COMMAND DID

"IBAMR Tutorial: UNC Edition" (2014), slides 4–5; the module list as printed in Dennis Evangelista's notes • historical, do not run

```
module list
  1) null
module initadd ibamr/3803
exit
# log in again
module list
Currently Loaded Modulefiles:
  1) null                 4) petsc/3.4            7) boost/1.53.0
  2) openmpi_intel/11.1   5) libmesh/0.9.3        8) ibamr/3803
  3) hdf5/1.8.5           6) silo/4.9.1
```

`module rm` removed one, `module initrm` removed it from the start-up files, and `module initlist` listed what later logins would load. The 2014 Kure guide lists "the basic module commands", `module [ add | avail | help | list | load | initadd | unload | initrm | show ]` (its glossary adds `rm` and `initlist`), and the rule: initadd "does not change your current session, only later logins."

- "Check that you only have the null module": a fresh account loaded nothing (`null`).
- `module add` loads a module for the current session; `module initadd` writes it into the shell's start-up files, so that it loads at every login. That is why the notes say to log out and back in.
- `module initadd ibamr/3803` brought the six modules IBAMR needed with it (the second listing, as printed in Dennis Evangelista's notes: seven entries after `null`, `ibamr/3803` included).

Today: `module load gnu13 openmpi5` inside each job script; IBAMR is your own autoibamr install, loaded with `enable.sh` (chapters 1–3).

"IBAMR Tutorial: UNC Edition" (2014), slides 4–5; Dennis Evangelista's notes (2014), section 3.1; "Using IBAMR Module on Kure" (May 2014); the 2016–17 module notes (chapter X2).

THE IBAMR/3803 MODULE (2014–2016), AND ITS RUNTIME ERROR

- IBAMR revision 3803 with the libraries of the last slide: “Feb. 2014 (?)” in the course Makefiles, and a deck dated “March 26, 2014” loads it (“You must first remove any other module”: `module remove openmpi___`, then `module add ibamr/3803`); used by the course until late 2016.
- With it, a job could stop at once with a missing Intel library; the 2014 notes give the error and the fix, typed before `bsub`:

“IBAMR Tutorial: UNC Edition” (2014), slide 11 • historical, do not run

```
/usr/mpi/intel/openmpi-1.4.1/bin/orted: error while loading shared
      ↪ libraries: libimf.so: cannot open shared object file: No
      ↪ such file or directory
export LD_LIBRARY_PATH=/nas02/apps/intel-11.1/cc/lib/intel64/
```

- The module’s OpenMPI was built with Intel’s compilers; its daemon `orted` needs Intel’s run-time library `libimf.so`; the `export` sets the library path to Intel’s folder alone (ending it with `:$LD_LIBRARY_PATH` would have kept the rest). Today: no `LD_LIBRARY_PATH` line; `enable.sh` sets the paths (chapter 2).
- Dennis Evangelista’s notes put the `export` between `make main2d` and `bsub`, and warn that with wrong library paths a job “will halt in about a minute”.
- The build side of this module (what it contained, how to build a newer IBAMR against it) is in chapter X2.

“IBAMR Tutorial: UNC Edition” (2014), slide 11; Dennis Evangelista’s notes (2014), section 3.3; the course Makefiles’ comments; “IBAMR Killdevil Module” (March 26, 2014), slides 1–4 and 9.

THE IBAMR/2016-11 MODULE (2016–2018)

"Getting Started on UNC cluster" (2017–18), slides 4–5 • historical, do not run

```
module initadd ibamr/2016-11
[<onyen>@killdevil-login2 ~]$ module list
  1) git/1.8.5.3         2) openmpi_gcc/4.8.1   3) python/2.6.5
     ↪       4) ibamr/2016-11
```

"Run your first IBAMR simulation" (January 2018; the module list retyped from its screenshot) • historical, do not run

```
module initadd ibamr/2016-11
module list
  1) openmpi_gcc/4.8.1   2) python/2.6.5   3) ibamr/2016-11
```

- The second IBAMR module ("Nov 2016" in the course Makefiles), built on the `openmpi_gcc/4.8.1` module (Open MPI with the GNU 4.8.1 compilers); the course used it on KillDevil ("We now only use the server killdevil", 2018).
- The 2018 handout added a link to UNC's "getting started on KillDevil" help page, and a note that most of its screenshots still showed Kure.
- Moving between `ibamr/3803`, `ibamr/2016-11` and a home build needed `module initrm` of the others first (chapter X2); the course's IB examples were moved to this module early in 2017 (chapter X3).

"IBAMR Tutorial: Getting Started on UNC cluster" (2017–18), slides 4–5 (prompt scrubbed); "Run your first IBAMR simulation" (2018), adapted from Nick Battista's 2014 notes.

KILLDEVIL BEFORE THE IBAMR MODULE (2012–2013)

ReadMe deck, slide 16 (2012) • historical, do not run

```
export FC=gfortran CC=gcc CXX=g++ F77=gfortran
module add openmpi_gcc; module initadd openmpi_gcc
```

forces deck, slide 2 (2013) • historical, do not run

```
export FC=gfortran CC=gcc CXX=g++ F77=gfortran
module initadd openmpi_gcc/4.4.6
```

ReadMe deck, slide 31; forces deck, slide 14 (two lines of prose left out) • historical, do not run

```
Put the folder with your specific code into the /netscr/onyen folder.
      ↪  Then cd into the folder.
make clean
make main3d
bsub -q week -n 16 mpirun ./main3d input3d
*-n 16 will run on 16 processors. I recommend running 16-32 (should
      ↪ be powers of 2).
Type bjobs -uonyen to check on the status.
```

- Before 2014 each user built IBAMR in the home folder (chapter X2) with KillDevil's GNU compilers and its `openmpi_gcc` module; the `export` line chose the GNU compilers.
- The run was already the LSF pattern of the next six years: copy the folder to `/netscr`, `make clean`, `make main3d`, `bsub`.
- `bjobs -uonyen`: `-u` with the user ID written right after it; "onyen" stands for your own.

The Miller lab's IBAMR ReadMe deck (2012), slides 16 and 31; the forces install deck (2013), slides 2 and 14.

YOUR OWN BUILD ON KILLDEVIL (2013): THE MAKEFILE, AND THE $HOME TRAP

"Running IBAMR on Killdevil" (2013), slide 2, as written (the home path as <o>/<n>/<user>) • historical, do not run

```
In the make file, change the paths to the following:
IBAMR_SRC_DIR = $HOME/sfw2/ibamr/IBAMR2
IBAMR_BUILD_DIR = $HOME/sfw2/ibamr/ibamr-objs-opt

In a lot of the example codes, one one set of paths might be
      ↪ commented out so that you can switch back and forth between
      ↪ killdevil and a THLinux box.

Warning - Something changed on killdevil so my home directory was
      ↪ messed up, and I had to put in the full path.
IBAMR_SRC_DIR = /nas02/home/<o>/<n>/<user>/sfw2/ibamr/IBAMR2
IBAMR_BUILD_DIR = /nas02/home/<o>/<n>/<user>/sfw2/ibamr/ibamr-objs-
      ↪ opt
```

The likely cause. In a Makefile, `$HOME` is not the shell's variable: `make` reads `$H` (a one-letter variable, empty) followed by `OME`, so the path becomes `OME/sfw2/...` (checked with GNU Make 4.3). Make needs `$(HOME)`, which the 2016-11 Makefiles write (in a commented-out line); the full path worked for the same reason.

"one one [*sic*] set of paths might be commented out": the habit that left every era's paths as comments in the course Makefiles (chapter X3). The build itself, in `$HOME/sfw2`, is chapter X2's.

Today: a `CMakeLists.txt`; no paths to write (chapter 3).

The instructor's "Running IBAMR on Killdevil" (created September 2013), slide 2 (the home path shown with placeholders).

POINTING A MAKEFILE AT THE MODULE

IBFE “Getting Started” decks, slides 6–7 • historical, do not run

```
Makefile flags (for ibamr/3803)
LIBSNEW = $(LIBS) -lnetcdf -lcurl
IBAMR_SRC_DIR = /nas02/apps/ibamr-3803/OFED-1.5.2/IBAMR2
IBAMR_BUILD_DIR = /nas02/apps/ibamr-3803/ibamr-objs-opt

Makefile flags (for ibamr/2016-11)
LIBSNEW = $(LIBS) -lnetcdf -lcurl
IBAMR_SRC_DIR = /nas02/apps/ibamr-2016-11/OFED-1.5.2/sfw/ibamr/IBAMR
IBAMR_BUILD_DIR = /nas02/apps/ibamr-2016-11/OFED-1.5.2/sfw/ibamr/ibamr-
      ↪ objs-opt
```

- Each example’s Makefile named the module’s source and build folders; IBFE examples also used `$(LIBSNEW)` “instead of the $(LIBS) flag” on their link lines.
- The course’s later Makefiles often keep earlier installs’ paths as comments (the 2016-11 ones list four install locations, three of them commented out), up to Dogwood’s and Arizona’s: chapter X3 dates old code by them; chapter X2 tells the story of the installs whose recipes survive.

Today: a `CMakeLists.txt` with `find_package(IBAMR)`; an old Makefile stops at its `make.inc` line (chapter 3).

The IBFE “Getting Started” decks (adapted from Alex Hoover’s 2014 tutorial; updated 2017–18), slides 6–7.

COPYING AN EXAMPLE UP: SCP, FILEZILLA, ZIP

Nick Battista's notes (2014), as written • historical, do not run

```
scp –r 1st_Simulation [your onyen]@kure.unc.edu:/netscr/[your onyen]
```

Dennis Evangelista's notes (2014) • historical, do not run

```
desktop$ scp FlowTank.zip <onyen>@kure.unc.edu:/netscr/<onyen>/
      ↪ FlowTank.zip
desktop$ ssh kure.unc.edu
<onyen>@kure-login$ cd /netscr/<onyen>/
<onyen>@kure-login$ unzip FlowTank.zip
<onyen>@kure-login$ cd FlowTank
```

- From the folder on your own computer that holds the example, `scp -r` copies a whole folder; "the colon, : , is necessary" between the host and the remote path.
- A slip: the notes' text has an en dash (`–r`, in red); the screenshot beside it shows the hyphen that works.
- Or zip the example, copy one file, and `unzip` it on the cluster; or drag it across in FileZilla to "your netscr/onyen directory on kure".

Today: `scp`, `rsync` or FileZilla to `filexfer.hpc.arizona.edu` (chapter 1).

Nick Battista's notes (2014), pages 4–5; Dennis Evangelista's notes (2014), section 3.2 (prompts with placeholders); "IBAMR Tutorial: UNC Edition" (2014), slide 8.

BUILDING ON THE LOGIN NODE: MAKE CLEAN, MAKE MAIN2D

Nick Battista's notes (2014), retyped; `make clean` from the 2014 Kure deck, slide 9 • historical, do not run

```
cd /netscr/<onyen>/1st_Simulation
make clean
make main2d
ls
```

"New files" after `make main2d`: `main2d`, `main.o`, `stamp-2d`, `update_target_point_positions.o`. The example folder also held `hier_data_IB2d`, `IB2d.log` and `viz_IB2d` after a run, the MATLAB scripts that made its geometry, and its `.target` and `.vertex` files.

- Building ran on the login node, as every course handout from 2012 to 2024 did: a few minutes of one core, tolerated where runs were not.
- `make clean` removed old object files first, so that a new module or a changed header was compiled from scratch (`make` recompiles a changed `.C` file by itself).
- `stamp-2d` is the Makefile's marker of the last dimension built (chapter X3).

Today: `cmake` in an `interactive` session or a short batch job; the build folder is `build` (chapter 3).

Nick Battista's notes (2014), pages 5 and 7 (Part I, steps 15–18 and 21) (the listing retyped from the screenshots, with the ONYEN replaced); "IBAMR Tutorial: UNC Edition" (2014), slide 9.

SUBMITTING WITH LSF: BSUB

2014 Kure deck, slide 9 • historical, do not run

```
make clean
make main2d
bsub -q week -n 2 mpirun ./main2d input2d
```

Nick Battista's notes (2014) • historical, do not run

```
bsub -q week -n 8 mpirun ./main2d input2d
Job <885471> is submitted to queue <week>.
```

IB talk (2016), page 74 • historical, do not run

```
module load ibamr/3803
make clean
make main2d
bsub -q week -n 4 mpirun ./main2d input2d
```

"bsub is the command to submit a simulation to the cluster. -q is a flag requesting a week queue (longest queue). -n is the number of processors. mpirun: multiple processors."

- `bsub` ("batch submit") took the command line itself; the notes used no job script. LSF printed the job's number, which the 2014 notes ask you to record with what the run was (the Reynolds number, the time step, the output interval).
- The counts in the notes: 2, 4 and 8 for 2D, 16 to 32 for 3D, "should be powers of 2".
- LSF e-mailed when a job finished or "halted abnormally".

Today: a job script with `#SBATCH` lines and `mpirun -np "$SLURM_NTASKS"`, submitted with `sbatch` (chapter 3).

"IBAMR Tutorial: UNC Edition" (2014), slide 9; Nick Battista's notes (2014), pages 5–6 (Part I, step 19); Dennis Evangelista's notes (2014), section 3.3; IB talk by Nick Battista and Laura A. Miller (7 September 2016), page 74.

WATCHING AND STOPPING LSF JOBS: BJOBS, BKILL

Nick Battista's notes (2014), the screenshot retyped • historical, do not run

```
[<onyen>@kure-login1 1st_Simulation]$ bsub -q week -n 8 mpirun ./main2d
      ↪ input2d
Job <885471> is submitted to queue <week>.
[<onyen>@kure-login1 1st_Simulation]$ bjobs
JOBID   USER    STAT  QUEUE    FROM_HOST    EXEC_HOST    JOB_NAME
      ↪ SUBMIT_TIME
885471  <onyen> RUN   week     kure-login1  8*public_ho  *d input2d  Aug 26
      ↪ 14:32
```

"Using IBAMR Module on Kure" (May 2014) • historical, do not run

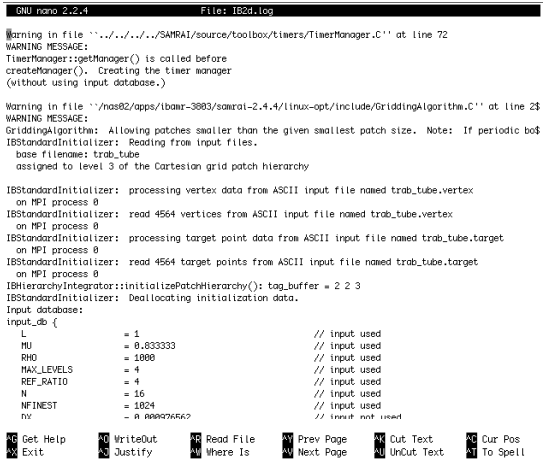
```
bsub - submit
bqueues - tells you about traffic on the queues
Can use debug queue to test if things work, int or day for interactive
bkill - kills a job
bjobs - tells you information about your current jobs. Use this to check on
      ↪ the status of your submitted jobs.
bjobs -l gives more information
bhist - tells you information about your job history
bpeek lets you see the output that is happening
Killdevil/Kure will send you an email when your simulation finishes running
      ↪ (or fails to run).
```

- `bjobs` works from any folder; it shows each job's state (`RUN` or `PEND`ing), its number, its processors and when it was submitted.
- `bjobs -u<onyen>` for one user; `bkill <jobid>` stops a job, `bkill 0` all of yours.

Today: `squeue --me`, `scancel <jobid>`, `sacct` for finished jobs (chapter 1).

Nick Battista's notes (2014), page 6 (Part I, step 20) (the screenshot retyped, the user name replaced); "IBAMR Tutorial: UNC Edition" (2014), slide 10; Dennis Evangelista's notes (2014), section 3.3; "Using IBAMR Module on Kure" (May 2014).

FOLLOWING THE LOG ON THE CLUSTER (1/2): NANO



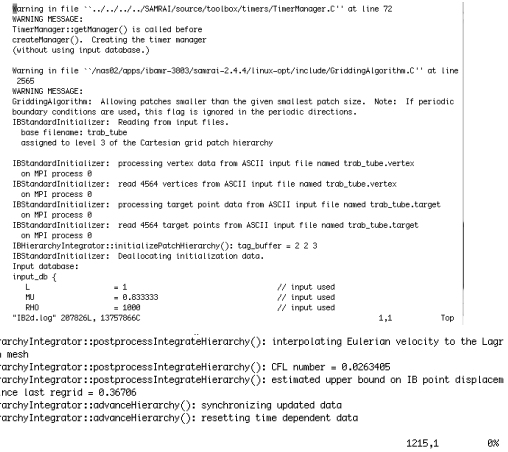
`nano IB2d.log` on Kure (2014): the top of the log, and nano's shortcut bar.

- `IB2d.log` reports, step by step, the time step, the simulation time, the Stokes solver's iterations and the CFL number.
- In nano: Ctrl+V a page down, Ctrl+Y a page up, Ctrl+W then Ctrl+V to the end and Ctrl+W then Ctrl+Y to the top (the notes write "Ctrl + w + v"), Ctrl+X to leave, answering `n` to "Save modified buffer?".
- "NOTE: For some reason as of August 2014, the nano editor cannot be used on Kill Devil, but it can be used on Kure."

Today: nano on Puma (chapter 1); `tail -f` on the log while a job runs (chapter 3).

Nick Battista, "Party time in on super computers!" (August 2014), pages 7–9 (Part I, steps 21–22 and the note after them), with his screenshots.

FOLLOWING THE LOG ON THE CLUSTER (2/2): VI



`vi IB2d.log` (2014): the top of the file, and line 1215 after `:1215`.

- `vi IB2d.log` opens the log; `:1215` jumps to line 1215 (the notes' example).
- `:wq` saves and quits, `:q` quits, `:q!` quits without saving.
- A slip: "To make a change in vi . . . hit the esc key. You can now write". It is the other way round: `i` starts typing, Esc stops it, and the colon commands work after Esc.

Today: vi on Puma (chapter 1).

Nick Battista, "Party time in on super computers!" (August 2014), pages 9–10 (Part I, step 24), with his screenshots; the vi slip is also in the 2018, 2020 and 2024 handouts.

GETTING RESULTS BACK (1/2): TAR AND SCP (2014)

Dennis Evangelista's notes (2014), section 3.4 • historical, do not run

```
<onyen>@kure-login$ tar -cvzf viz_IB2d.tar.gz viz_IB2d/
<onyen>@kure-login$ exit
desktop$ scp <onyen>@kure.unc.edu:/netscr/<onyen>/FlowTank/viz_IB2d.
      ↪ tar.gz .
desktop$ tar -xvf viz_IB2d.tar.gz
desktop$ visit &
```

Nick Battista's notes (2014), as written • historical, do not run

```
scp -r [your onyen]@kure.unc.edu:/netscr/[your onyen]/1st_Simulation
      ↪ Finished_Simulation
```

- Copy the whole folder back "at least within a week" (`/netscr` "recycles files"), to an external drive: 500 MB to 4 GB per simulation; data can be copied while the run is still going.
- One compressed file copies faster than thousands of small ones: `tar -cvzf` on the cluster, `tar -xvf` at home.
- Then VisIt on your own computer: `lag_data.visit` for the structure, `dumps.visit` for the mesh, velocity and vorticity.

Today: `rsync` or `scp` from `/xdisk`, then remove what you copied (chapters 1 and 3).

Dennis Evangelista's notes (2014), section 3.4 (prompts with placeholders); Nick Battista's notes (2014), part II (en dash as written).

GETTING RESULTS BACK (2/2): A STUDENT'S COPY SCRIPTS

"Running many simulations Part 2" (2018), slides 19–20, condensed: comment lines and one `echo` line left out; user names and folders replaced • historical, do not run

```
# This script downloads simulation results from KILLDEVIL cluster to my
      ↪ computer
echo "Downloading..."
scp -r <onyen>@killdevil.unc.edu:/netscr/<onyen>/<run_folder>/ /path/to
      ↪ /<results_folder>/

# Uploads <run_folder> to KILLDEVIL cluster from my computer
scp -r /path/to/<inputs_folder>/<run_folder> <onyen>@killdevil.unc.edu:/
      ↪ netscr/<onyen>/
```

- Two short scripts, one `scp` line each ("The active line of code"), with comments that explain `scp` and `-r`, so that a run folder went up and its results came down with one command.
- Copying the run folder back whole brings the restart files and the logs with the pictures.

Today: the same scripts with `filexfer.hpc.arizona.edu` and `/xdisk/<pi>/<netid>`, or `rsync -av` (chapter 1).

"7-IBAMR- Running many simulations Part 2" (March 2018), slides 19–20: scripts by a student of the course (not named), with user names and folders replaced.

3D RUNS ON KURE AND KILLDEVIL

"IBAMR Tutorial: UNC Edition" (2014), slide 10 • historical, do not run

```
make clean
make main3d
bsub -q week -n 16 mpirun ./main3d input3d
```

"this takes MUCH longer".

TubeHeart "Read me" (February 2017) • historical, do not run

```
These examples use a lot of memory. Please submit with this
      ↪ command
bsub -q week -n 32 -M 10 ./main3d input3d
```

No `mpirun`: LSF reserves 32 slots and one `main3d` runs (the Slurm case is slide 69). `-M` is LSF's memory limit (per process, unless the site made it per job); the note gives no unit, and LSF's default, KB, would make `-M 10` absurd for "a lot of memory", so UNC's unit was probably GB.

- The same example built in 3D (`main3d`, `input3d`), on 16 processors; the 2012 notes recommend 16 to 32, "powers of 2".
- A 3D run could hold its cores for days in the `week` queue, LSF's longest.
- `make clean` came first "for good measure" (the 2012 notes); the Makefile itself cleaned the shared object files when the dimension changed, using its `stamp-2d` and `stamp-3d` markers (chapter X3).

Today: `main3d` is its own CMake target; a long run uses restarts (chapter 3).

"IBAMR Tutorial: UNC Edition" (2014), slide 10; the Miller lab's ReadMe deck (2012), slide 31; the TubeHeart "Read me" (2017).

MANY RUNS AT ONCE UNDER LSF (2018) (1/6): THE PLAN AND THE TEMPLATE

"Goal": run many simulations that vary parameters of `input2d`. Three pieces: a template `input2d`, a script `setinput2d.sh` that writes numbered input files, and a script that submits them all.

`allpara.txt`: three tab-separated columns, the Womersley number WO (changed through the viscosity), PAMP (the percent contraction, not used) and FREQ (the pumping frequency).

slide 4, "template-input2d", the lines that matter (tabs shown as spaces) • historical, do not run

```
FRE = 1.25          //pumping frequency (Hz)
WO = 5.05           // desired Womersley number,original
      ↪ calculation
MU = 2.0*PI*FRE*RHO*D*D/(4.0*WO*WO)     // fluid dynamic viscosity
      ↪  (N s / m^2) to yield desired Wo, original calculation
```

The template also sets `L = 1.0`, `D = 0.1`, `RHO = 1000.0`, `PI = 4.0*atan(1);` and keeps `//MU = 10`. Its output settings (slide 5): `viz_writer = "VisIt"`, `viz_dump_interval = 1000`, `viz_dump_dirname = "viz_IB2d"`, `visit_number_procs_per_file = 1`, `restart_interval = 0`, `restart_write_dirname = "restart_IB2d"`, `data_dump_interval = 0`, `data_dump_dirname = "hier_data_IB2d"`.

The viscosity is computed from the Womersley number, $\mathrm{Wo}^2 = \omega D^2/(4\nu)$ with $\omega = 2\pi f$ and $\nu = \mu/\rho$: setting WO and FRE sets MU. Chapter 3 quotes part of the generator and its three faults; these six slides keep the whole of it, as chapter 3 promises.

The instructor's "7-IBAMR- Running many simulations Part 2: Sample code" (March 2018), slides 2–5.

MANY RUNS AT ONCE UNDER LSF (2018) (2/6): THE GENERATOR, ITS START

`setinput2d.sh`, slide 6 • historical, do not run

```
#!/bin/bash

# Separate main parameter file into three files
# Take the first column which lists the Womersley number (which
      ↪ is varied by changing viscosity).
cut -f 1 allpara.txt > Wo.txt
# Take the second column which lists the percent amplitude of
      ↪ contraction.
# cut -f 2 allpara.txt > pamp.txt
# Note that in this case the percent amplitude of contraction is
      ↪  not used.
# Take the third column which is the frequency.
cut -f 3 allpara.txt > Freq.txt
```

slides 7–8 • historical, do not run

```
# Count number of lines in files
numlines=$(grep -c "^" allpara.txt)
# "^" basically reads a line that starts with anything (e.g.
      ↪ reads all lines).
# -c is an option to count.
# allpara.txt is the text files to read.

# initialize variables
WO=0
Freq=0

# For loop that will write files, count from 1 to the number of
      ↪  lines
for i in `seq 1 $numlines`;
do
```

“7-IBAMR- Running many simulations Part 2: Sample code” (March 2018), slides 6–8.

MANY RUNS AT ONCE UNDER LSF (2018) (3/6): THE LOOP SETS WO AND FRE

`setinput2d.sh`, slides 9–10 (inside the loop) • historical, do not run

```
# Writes file to replace line with Wo
# Go to line 7 in template-input2d (which in the input2d template is the WO),
# Set that line to WO = "'"var"'"   // Womersley number
# Copy that file to input2dw${i}
awk -v var="$WO" 'NR==7 {$0="WO = "'"var"'"   // Womersley number"} 1' template-input2d > input2dw${i}

# Sets Freq based on i
# Read line i from the file Freq.txt and assign that value to Freq.
Freq=$(awk -v var="$i" 'NR==var' Freq.txt)
# Writes file to replace line with Freq
# Go to line 5 in the template, set that line to FRE = "'"var"'"   // Pumping Frequency (Hz)
# Copy that file to input2dw${i}f${i}
awk -v var="$Freq" 'NR==5 {$0="FRE = "'"var"'"   // Pumping Frequency (Hz)"} 1' input2dw${i} > input2dw${i}f${i}
```

Each `awk` line replaces one whole line of the input file, chosen by its number (`NR==7`, `NR==5`), and writes a new file. Freq is read from line `$i` of `Freq.txt`; WO is not read at all (slide 44).

“7-IBAMR- Running many simulations Part 2: Sample code” (March 2018), slides 9–10.

MANY RUNS AT ONCE UNDER LSF (2018) (4/6): LOG AND OUTPUT FOLDERS, AND THE RUN LENGTH

`setinput2d.sh`, slides 11–12 • historical, do not run

```
# Edits input2d to create different IBlog and visit files and folders
awk -v var="$i" 'NR==201 {$0="   log_file_name = \"runs/IB2d.log"'"var"'"\"                //"} 1' input2dw${i}f${i} > input2dw${i}
      ↪ f${i}l${i}
awk -v var="$i" 'NR==207 {$0="   viz_dump_dirname = \"runs/viz_IB2d"'"var"'"\"              //"} 1' input2dw${i}f${i}l${i} >
      ↪ input2dw${i}f${i}l${i}y${i}

# The code below changes the simulation time based on the pumping frequency
test=$Freq
# echo $test
if [ $(echo " $test <= 0.70" | bc) -eq 1 ]
then
    cat template-input2d |  awk 'NR==24 {$0="END_TIME =      3.5               // Final Simulation Time (s)"} 1' input2dw${i}f${i}
      ↪ l${i}y${i} > input2dw${i}f${i}l${i}y${i}t${i}
elif [ $(echo " $test < 1.0" | bc) -eq 1 ]
then
    cat template-input2d |  awk 'NR==24 {$0="END_TIME =       3.0              // Final Simluation Time (s)"} 1' input2dw${i}f${i}
      ↪ l${i}y${i} > input2dw${i}f${i}l${i}y${i}t${i}
else
    cat template-input2d |  awk 'NR==24 {$0="END_TIME =       2.5               // Final Simulation Time (s)"} 1' input2dw${i}f${i}
      ↪ l${i}y${i} > input2dw${i}f${i}l${i}y${i}t${i}
fi
```

The run length follows the frequency: 3.5 s up to 0.70 Hz, 3.0 s below 1.0 Hz, 2.5 s otherwise, so that each run covers enough cycles.

“7-IBAMR- Running many simulations Part 2: Sample code” (March 2018), slides 11–12.

MANY RUNS AT ONCE UNDER LSF (2018) (5/6): THE END OF THE LOOP

`setinput2d.sh`, slide 13; the closing `done` from slide 8 • historical, do not run

```
# Cleans up folder
rm input2dw${i} input2dw${i}f${i} input2dw${i}f${i}l${i} input2dw${i}f${i}l${i}y${i}
mv input2dw${i}f${i}l${i}y${i}t${i} input2d${i}

# Prints out END_TIME and FRE for a check
echo $i
grep "END_TIME =" input2d${i}  && grep "FRE ="  input2d${i}
done
```

Each step writes a new file named after the steps so far (`w`, `f`, `l`, `y`, `t`), and the last is renamed `input2d${i}`. The final `done` (slide 8, which shows the loop as `do ... done`) closes the loop opened on slide 40.

“7-IBAMR- Running many simulations Part 2: Sample code” (March 2018), slide 13.

MANY RUNS AT ONCE UNDER LSF (2018) (6/6): SUBMITTING THEM, AND THE FAULTS

slide 14, “Run simulations” • historical, do not run

```
#!/bin/bash

startrun=31
endrun=40
mkdir parameters-files

# for i in "${!array}";
for i in `seq $startrun $endrun`;
do
   # run MPI program
   #Assumes all input2d files are in a folder called
      ↪ input2d-files
   bsub -q week -n 16 mpirun ./main2d input2d-files/
      ↪ input2d${i}
done
```

The faults (chapter 3 points out the first three):

- WO is never read from `Wo.txt` (there is no `WO=$(awk ... Wo.txt)` line, as there is for Freq), so every file gets `WO = 0` and the formula for MU divides by zero.
- The generator writes `input2d${i}` in the current folder; the runner reads `input2d-files/input2d${i}`; the `parameters-files` folder it makes is never used.
- Fixed line numbers (`NR==7`, `5`, `201`, `207`, `24`) break silently when the template changes.
- Neither script makes the `runs/` folder named in the log and visualization lines; runs 31–40 are hard-coded; `cat template-input2d |` before an `awk` that reads a named file does nothing; “Simluation”.

Today: replace values by name and submit a Slurm job array (chapter 3).

“7-IBAMR- Running many simulations Part 2: Sample code” (March 2018), slide 14; the faults checked against slides 6–14 (October 2026).

THE FIRST-SIMULATION HANDOUTS (1/2): UNC, 2014–2018

date	handout	machine	IBAMR	runs with	runs in
May 2014	“Using IBAMR Module on Kure - Instructions for Undergraduates”	Kure, KillDevil	`ibamr/3803`	`bsub -n 2`	`/netscr/onyen`
May 2014	Dennis Evangelista’s notes	Kure	`ibamr/3803`	`bsub -n 2`	`/netscr/<onyen>`
May 2014, Jan 2015	“IBAMR Tutorial: UNC Edition” (MATH 769, Lecture 3)	Kure	`ibamr/3803`	`bsub -n 2`	`/netscr/<onyen>`
Aug 2014	Nick Battista, “Party time in on super computers!”	Kure, KillDevil	`ibamr/3803` (in its log)	`bsub -n 8`	`/netscr/<onyen>`
2017–18	“Getting Started on UNC cluster”	KillDevil	`ibamr/2016-11`	`bsub -n 2`	`/netscr/<onyen>`
Jan 2018	“1 - Get killdevil account and software” (the account and software steps)	KillDevil	–	–	–
Jan 2018	“Run your first IBAMR simulation” (adapted from the 2014 notes)	KillDevil	`ibamr/2016-11`	`bsub -n 8`	`/netscr/<onyen>`

The handouts named in the table.

THE FIRST-SIMULATION HANDOUTS (2/2): 2018–2026

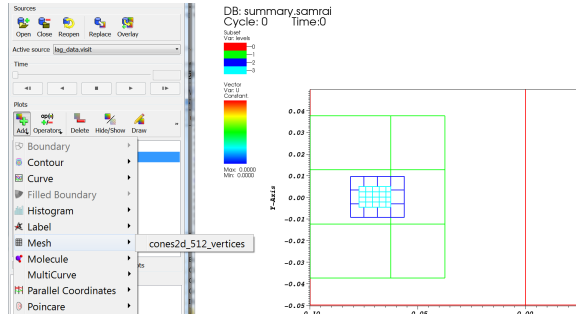
date	handout	machine	IBAMR	runs with	runs in
Apr 2018	“2-Run your IBAMR simulation on Dogwood”	Dogwood	`ibamr/2018-03`	`sbatch`	`/21dayscratch/...`
May 2020	“Get dogwood account and run a simulation” (adapted again)	Dogwood	`ibamr/2018-03`	`sbatch`	`/21dayscratch/...`
July 2024	“Get HPC account and run a simulation” (UA Lecture 1)	Puma, Ocelote, El Gato	a group install	`sbatch`	`/xdisk/<group>/<netid>`
2025–26	this series, chapters 1–3	Puma	autoibamr	`sbatch job.slurm`	`/xdisk/<pi>/<netid>`

One handout, rewritten for each machine: the 2014 text survives into 2020 with “killdevil” and “netscr” left in a Dogwood guide, and the 2020 guide still shows the 2014 Kure screenshots. The May 2014 Kure guide is the source of the home, module, `/netscr` and `bsub` slides of the Kure deck of the same month, which links to it, and the 2018 KillDevil and Dogwood notes are the sources of the 2020 guide. Each change of machine changed four lines: the host, the IBAMR module, the scheduler command and the scratch folder.

The handouts named in the table.

THE FLOWTANK FIRST RUN (2014)

- The May 2014 notes started everyone on one example, the flow tank: a channel “10x10x20 cm with parabolic inflow and outflow”, with a triangle (2D) or a cone (3D) held in place by target points; its `.vertex` and `.target` files came from `generate_mesh2d.m` and `generate_mesh3d.m`.
- `FlowTank.zip` was uploaded to `/netscr/<onyen>`, built and submitted as on the last slides; the results came back as `viz_IB2d` (the deck says “viz2d (or viz3d)”) for VisIt.
- In VisIt: `lag_data.visit` for the Lagrangian points, `dumps.visit` (the deck also writes “Dump.visit”) for the Eulerian fields; Add, Vector, U and Add, Pseudocolor, Omega; Add, Mesh, `cones2d_512_vertices` for the triangle’s points.



VisIt 2.x (2014): adding the triangle’s points as a Mesh plot over the patch levels of the flow tank.

Today: IB/03 (a tethered beam in a channel) and IBFE/10 (a plate in a 3D flow tank); VisIt is chapters 26–27.

“IBAMR Tutorial: UNC Edition” (May 2014), slides 7–8 and 13–17, with its screenshot (slide 17); Dennis Evangelista’s notes (2014), section 3.2.

IB2D ON KURE AND KILLDEVIL (2016)

"Getting Running on IB2D and IBAMR" (2016), slide 2 • historical, do not run

```
# "Ssh into kure. Cd into the directory where you want to
#  download ib2d. In the command window type:"
module add git/1.8.5.3
git clone https://github.com/nickabattista/IB2d.git
```

IB talk (2016), page 67, "How to run IB2d" • historical, do not run

```
ssh [your-onyen]@kure.unc.edu
module load matlab
# change to the example folder; edit input2d
bmatlab main2d.m
```

- IB2d, Nick Battista's MATLAB and Python immersed boundary code, ran on the clusters as a MATLAB batch job (next slide); `git` itself was a module.
- Output: the flow and Lagrangian positions in `viz_IB2d`; "Force data is in the `hier_IB2d_data` folder". (The IBAMR examples' folder of the same deck is `hier_data_IB2d`: two programs, two names; both are right.)
- "Be sure to module add ibamr/3803. May not work if matlab module is loaded" for IBAMR on the same machines (slide 50).

Today: `git clone` on Puma with no module, MATLAB as a module in a job script (chapter 1); IB2d itself is Part II.

"Getting Running on IB2D and IBAMR" (MATH 768, 2016), slides 2–4, 7 and 9; IB talk by Nick Battista and Laura A. Miller (7 September 2016), page 67.

MATLAB JOBS UNDER LSF: BMATLAB

"Getting Running on IB2D and IBAMR" (2016), slides 5–6 • historical, do not run

```
module add matlab
# with a window (from your computer):
ssh -X onyen@kure.unc.edu
module add matlab
xmatlab
# a serial batch job:
bmatlab matlabjob.m
# or the same job with bsub, on one line:
bsub matlab -nodisplay -nosplash -singleCompThread -r matlabjob -
      ↪ logfile matlabjob.out
```

- `bmatlab` was a UNC script that submitted a MATLAB file as a one-core LSF job, "the easiest way to submit a serial Matlab job" on Kure and KillDevil.
- With `bsub matlab ... -r matlabjob` the `.m` is left off the file name after `-r`; the output goes to `matlabjob.out` in the working folder.
- `ssh -X` forwarded MATLAB's window to your own screen (`xmatlab`).

Today: a Slurm script with `module load matlab` and `matlab -nodisplay -nosplash < main2d.m` (chapter 1).

"Getting Running on IB2D and IBAMR" (2016), slides 5–6 (paraphrasing UNC's help pages of the time); the IB talk (2016), page 67.

MODULE CLASHES THE NOTES WARNED ABOUT

- 2014: “You must first remove any other module” before `module add ibamr/3803`; “You may also need to delete manually in your .profile, .bashrc, and .bash_profile files in your home directory” (the `initadd` lines).
- 2016, IBAMR and MATLAB: “Be sure to module add ibamr/3803. May not work if matlab module is loaded.”
- 2016, VisIt: “You will want to module rm ibamr/3803 as there is a conflict”; then `module add visit`.
- 2017–18, VisIt with the 2016-11 module: `module rm ibamr/2016-11` before `module add visit/2.12.3` (next slides).
- The likely cause: each module put its own libraries first on the library path (HDF5, Python, compilers), and two modules wanted different versions.

“Analyzing your IBFE simulation” (2018), slide 8, condensed • historical, do not run

```
[<onyen>@killdevil-login2 ~]$ module list
  1) git/1.8.5.3  2) openmpi_gcc/4.8.1  3) python/2.6.5  4)
      ↪ ibamr/2016-11
[<onyen>@killdevil-login2 ~]$ module rm ibamr/2016-11
[<onyen>@killdevil-login2 ~]$ module add visit/2.12.3
```

Today: load only what a job needs, in the job script itself; a VisIt job loads `visit` and nothing of IBAMR’s (chapter 31).

“IBAMR Killdevil Module” (2014), slide 2; “Getting Running on IB2D and IBAMR” (2016), slides 9–10; “Analyzing your IBFE simulation” (2018), slide 8; “Visit from command line and Python” (2018), slide 3.

VISIT ON KILLDEVIL (1/3): THE MODULES

- Two VisIt modules: `visit` (VisIt 2.5.2) and `visit/2.12.3`. Scripts had to match: “record your python commands in visit 2.5.2. You can use later versions of visit, but you may need to debug your code. This older version of visit may also be incompatible with the newest Mac OS.”
- “Warning to user! The visit installed on killdevil cannot read .ex2 files. You will need to download the .ex2 files, export them as .vtk files, and then create a script that reads in vtk.”
- With a window: “If you ssh -X into killdevil, typing visit then brings up the GUI.”
- Paths recorded on a laptop or desktop (`/Users/<user>/Desktop/...`) had to be changed to the cluster’s (`localhost:/nas/...`) before a script ran there: “I recommend doing a search to find all paths.”

> Today: Puma’s `visit` module is VisIt 3.4.2, or your own VisIt 3.5.0; VisIt 3 reads ExodusII files, and VisIt 3.2 and later run Python 3 scripts (chapter 31).

“Resampling AMR grids” (2017–18), slide 2; “Analyzing your IBFE simulation” (2018), slides 6, 9 and 15; “Getting Running on IB2D and IBAMR” (2016), slide 10.

VISIT ON KILLDEVIL (2/3): SCRIPTS WITHOUT A WINDOW

"Visit from command line and Python" (2018), slide 3, condensed • historical, do not run

```
[<onyen>@killdevil-login1 <run_folder>]$ module rm ibamr/2016-11
[<onyen>@killdevil-login1 <run_folder>]$ module add visit/2.12.3
[<onyen>@killdevil-login1 <run_folder>]$ visit -cli -nowin
Running: cli2.5.2 -nowin
Running: viewer2.5.2 -host 127.0.0.1 -port 5600 -noint -nowin
Python 2.6.4 (r264:75706, Aug 23 2012, 13:48:37)
[GCC Intel(R) C++ gcc 4.1 mode] on linux2
>>> Source("heart_script.py")
```

slide 4, "Alternative method (easier)" • historical, do not run

```
[<onyen>@killdevil-login1 <run_folder>]$ visit -cli -nowin -s
      ↪ heart_script.py
Running: cli2.12.3 -nowin -s heart_script.py
Running: viewer2.12.3 -nowin -noint -host 127.0.0.1 -port 5600
Running: mdserver2.12.3 -host 127.0.0.1 -port 5600
Running: engine_ser2.12.3 -host 127.0.0.1 -port 5600
```

- `visit -cli -nowin` runs VisIt's Python interface with no window; `Source()` runs a recorded script inside it, and `-s` runs it from the command line.
- A puzzle in the transcript: after `module add visit/2.12.3` the first run reports VisIt 2.5.2 (the default module's), the second 2.12.3. The 2017 resampling deck (slide 17) has the same transcript after `module add visit`, the 2.5.2 default: the 2018 slide most likely edited the module line of that older transcript.
- Both ran on the login node, as the notes did then.

Today: `visit -cli -nowin -s script.py` with the full path, in a batch job (chapter 31).

"Visit from command line and Python" (2018), slides 1–4 (prompts with placeholders; the script name as on the slide).

VISIT ON KILLDEVIL (3/3): BUILDING YOUR OWN (2018)

"Visit from command line and Python" (2018), slides 7–10, condensed • historical,

do not run

```
# get build_visit2_12_3 (VisIt 2.12.3 releases page)
# put it in e.g. $HOME/sfw/visit
chmod 755 build_visit2_12_3
./build_visit2_12_3 --mesa        # answer "yes"
# then, in the visit2.12.3/src folder:
make package
# untar, then install:
svn_bin/visit-install VERSION ARCH INSTALL_DIR_PATH
svn_bin/visit-install 2_12_3 linux-x86_64 $HOME/sfw/visit_12_2_3
# run without the GUI:
cd $HOME/sfw/visit_12_2_3/bin
./visit -cli -nowin
```

- Your own VisIt 2.12.3 (the version of the `visit/2.12.3` module; the default `visit` was 2.5.2), built from source with `build_visit` and Mesa (software rendering, for a machine with no screen), then installed with `visit-install`.
- Slips: the heading says "Install your own version of visit (dogwood)" over KillDevil steps; the version argument is written `2_12_3`, where `visit-install` expects `2.12.3`, and the install folder `visit_12_2_3` transposes the version (harmless, but confusing); the slides print a full home path with a user name (replaced here by `$HOME`).
- The rest of the deck (recording scripts in the Commands window, saving frames in a loop) is chapters 31–32.

Today: a VisIt release for Rocky Linux 9, installed with `visit-install` (chapter 31).

"Visit from command line and Python" (2018), slides 7–10 (home paths replaced; the steps condensed, the commands as written).

WHEN A JOB “FINISHED” IN A SECOND, AND OTHER KILLDEVIL TROUBLES

> “the parallelization has been sort of buggy recently. . . if you submit a job and it finishes within a fraction of a second. . . the simulation DID NOT finish. . . try resubmitting the job OR run it on a single node, e.g., change to a 1” (September 2016): that is `-n 1`, one process.

The other troubles of the notes: the `libimf.so` error of the 3803 module (slide 25); module clashes (slide 50); `nano` not working on KillDevil in August 2014 (slide 34); a home path that “was messed up” in a Makefile (slide 28); `/netscr` files vanishing after 21 days (slide 22).

- A job that ends at once has failed at start-up: a missing library, a bad input line, an MPI launch problem. When resubmitting helped, the failure was probably in a node, not in the job.
- The first thing to read is the job’s output and error file; LSF e-mailed a summary.

Today: read the `.out` file and `sacct`; chapter 3 lists the common first errors, and chapter 16 the ones that come later.

KURE AND KILLDEVIL, THEN AND NOW

	Kure, KillDevil (2012–2018)	Puma (2025–26; chapters 1–3)
account	ONYEN Services, “subscribe to services”	HPC portal; a PI sponsors you into their group
log in	`ssh <onyen>@kure.unc.edu` (VPN off campus)	`ssh <netid>@hpc.arizona.edu`, then `shell` (no VPN for SSH)
run folder	`/netscr/<onyen>`, 21 days	`/xdisk/<pi>/<netid>`, 150 days (+150)
software	`module initadd ibamr/3803` (or `2016-11`)	autoibamr in your home; `module load gnu13 openmpi5`; `enable.sh`
build	a Makefile with `IBAMR_SRC_DIR`; `make main2d` on the login node	CMake; build in an `interactive` session or a job
submit	`bsub -q week -n 8 mpirun ./main2d input2d`	`sbatch job.slurm` with `mpirun -np "$SLURM_NTASKS"`
watch, stop	`bjobs`, `bkill`	`squeue --me`, `scancel`, `sacct`
log	`nano` or `vi IB2d.log`	the log and `.out` file, `tail -f`
results	`tar`, `scp` back within a week	`rsync` or `scp`; `/groups` or your own drive
VisIt	modules 2.5.2 and 2.12.3; no ExodusII	module 3.4.2 or your own 3.5.0

DOGWOOD AND LONGLEAF AT A GLANCE (2018–2020)

- Two UNC clusters after KillDevil, sharing a home folder and a software tree (`/nas/longleaf/...`): Dogwood, built for large parallel jobs that talk across many nodes (the course's IBAMR runs), and Longleaf, for data-heavy work on one node at a time (the lecture's COMSOL run, for instance).
- The scheduler was Slurm: "Dogwood uses the SLURM cluster management and scheduling software" (2018; the 2019–20 lecture made it "Dogwood/longleaf" and added a note for the course: "*killdevil used LSF").
- IBAMR came as a module again: `ibamr/2018-03` (March 2018), then `ibamr/0.6.0` (June 2020).
- Storage: scratch folders (`/21dayscratch` on Dogwood, emptied after 21 days; `/pine/scr` on Longleaf, whose clean-up the notes do not state), a home folder, and mass storage (`~/ms`, up to 1 TB) for keeping results.

> The course slides of 2018–20 copied their descriptions from UNC's Research Computing pages; this chapter rewrites them. Puma today plays both roles: one cluster for parallel IBAMR runs and for single-node work (chapter 1).

"2-Run your IBAMR simulation on Dogwood" (April 2018), slides 2 and 4, repeated with Longleaf added in Lecture 8, "COMSOL workflow, export and run remotely" (2019–20), slides 18 and 20; "Get dogwood account and run a simulation" (2020); the course Makefiles' comments; IBFE lecture 5 (June 2020). Retirement dates are not stated in the course's files.

WHERE THE DOGWOOD NOTES CAME FROM

- January 2018: “1 - Get killdevil account and software” gave the account and software steps; by May 2020 they were the Dogwood guide’s steps 1–7 (the guide’s Drive copy dates from November 2019), with “killdevil” changed to “dogwood” in most places.
- April 2018: “2-Run your IBAMR simulation on Dogwood” (27 slides; created April 2018, edited until May 2020) introduced Dogwood, Slurm, the `ibamr/2018-03` module and the first job scripts; it also holds the debugging tips (slides 13–16), added on a date not recorded. It is the original of the Slurm slides that the 2019–20 COMSOL lecture repeats (slides 18–26 of that lecture are its slides 2–10, with Longleaf added).
- July 2019: “Debugging Tips”, the same tips (= slides 13–16 of the 2018 deck; made in three minutes, so a copy; slide 72).
- November 2019 – May 2020: “Get dogwood account and run a simulation”, revised in May 2020 for the online summer series.

So the Slurm material of the next slides comes from a deck begun in April 2018, even where it is quoted from a later copy.

GETTING A DOGWOOD ACCOUNT (2020)

- ONYEN Services, “Subscribe to Services”, Dogwood. The page could answer: “You are ineligible for the following services . . . KillDevil Cluster: missing `'(LIVE|EXCH)'` prerequsite” [*sic*].
- If it did, an e-mail to UNC Research Computing saying so, with: your ONYEN, your UNC e-mail address, full name, campus address, a campus phone and a number where you can be reached while running jobs, department, your faculty sponsor’s (PI’s) name and ONYEN if known, and a description of the work; and “do not request to be added to a group”.
- These steps are the January 2018 KillDevil handout’s, with “killdevil” changed to “dogwood” in most places (slide 57).
- The guide of May 2020 then installs the desktop tools: an SSH program (Mac Terminal or PuTTY, port 22), VPN, FileZilla, VisIt, a plain-text editor, MATLAB.

> Today: the UA HPC portal, a PI group that sponsors you, and the group’s allocation on Puma (chapter 1). Personal details go to the university’s forms, never into shared notes.

“Get dogwood account and run a simulation” (the instructor, May 2020; its second part adapted from Nick Battista’s 2014 notes), Step 1 and Steps 2–7; “1 - Get killdevil account and software” (January 2018).

OFF CAMPUS: VPN AND DUO (2019–20)

- From off campus, the VPN client first, then `ssh` or FileZilla; the 2018 slides (repeated in the 2019 lecture) warned that a Cisco client already set up for another VPN "may not let you switch between the two".
- With it came Duo, UNC's two-step login on a phone.
- The May 2020 guide pointed to UNC's VPN help page and the campus software site for the client.

> Today: no VPN for SSH or Open OnDemand at Arizona; two-step login (Duo) at the bastion host; the HPC VPN only for programs that cannot pass the bastion (chapter 1).

"2-Run your IBAMR simulation on Dogwood" (2018), slide 3 = Lecture 8 (2019–20), slide 19; "Get dogwood account and run a simulation" (2020), Step 3 and Part I, item 9.

STORAGE ON DOGWOOD AND LONGLEAF

2019–20 lecture and 2020 guide (<onyen> for the user ID) • historical, do not run

```
# Dogwood scratch (run here; emptied after 21 days)
cd /21dayscratch/scr/o/n/<onyen>/
# Longleaf scratch
/pine/scr/o/n/<onyen>
# Longleaf home
/nas/longleaf/home/<onyen>
# mass storage in your home: up to 1 TB
cd ~/ms
```

- In `scr/o/n/<onyen>`, `o` and `n` stand for the first two letters of your ONYEN.
- "whatever you put in this directory is erased after 21 days"; move results to mass storage, "preferably in tar or zip format".
- Mass storage (`~/ms`): "Do not run programs out of this directory."
- Leftovers in the 2020 guide: "To run simulations we must change directories to our netscr directory" (KillDevil's folder) before the Dogwood path.

Today: `/xdisk/<pi>/<netid>` for runs; results to your own computer or an external drive; `/groups/<pi>` for the group's shared files (chapter 1).

"2-Run your IBAMR simulation on Dogwood" (2018), slides 2 and 4 (with Longleaf added in Lecture 8, 2019–20, slides 18 and 20); "Get dogwood account and run a simulation" (2020), pages on storage; G. Hobson's 2020 VisIt tutorial (the Longleaf home).

MOVING FILES TO AND FROM DOGWOOD

"Get dogwood account and run a simulation" (2020), as written • historical, do not run

```
scp –r 1st_Simulation [your onyen]@dogwood.unc.edu: /21dayscratch/scr/
      ↪ o/n/onyen
scp –r [your onyen]@dogwood.unc.edu:/21dayscratch/scr/o/n/[your onyen
      ↪ ]/1st_Simulation Finished_Simulation
```

the first line, as it should read • historical, do not run

```
scp -r 1st_Simulation <onyen>@dogwood.unc.edu:/21dayscratch/scr/o/n/<
      ↪ onyen>
```

- The 2014 commands with the new host and scratch path, and two slips: the en dash in `–r`, and a space after the colon, which makes the scratch path a separate argument: `scp` then takes it as the destination on your own computer, and the remote home folder as a second source.
- FileZilla works too; the guide's heading still says "SFTP (moving files between local computer and killdevil)".
- Copy back "at least within a week", to an external drive (the 2014 advice, unchanged).

Today: `scp`, `rsync` or FileZilla to `filexfer.hpc.arizona.edu` (chapter 1).

"Get dogwood account and run a simulation" (2020), Step 4, Part I (item 15) and Part II (en dashes and placeholders as written).

THE DOGWOOD IBAMR MODULES: 2018-03 AND 0.6.0

2020 guide (prompt scrubbed) • historical, do not run

```
[<onyen>@dogwood-login1 769_examples]$ module add ibamr/2018-03
[<onyen>@dogwood-login1 769_examples]$ module list
Currently Loaded Modules:
  1) openmpi_2.0.3/gcc_4.8.5   2) ibamr/2018-03
```

IBFE lecture 5, “Plans for June” (1 June 2020), slide 3 • historical, do not run

```
To load the old module:
module add ibamr
module list
Currently Loaded Modules:
 1) openmpi_2.0.3/gcc_4.8.5   2) ibamr/2018-03
To load the new module
module add ibamr/0.6.0
module list
Currently Loaded Modules:
  1) openmpi_2.0.3/gcc_4.8.5   2) ibamr/0.6.0
To remove a module
module rm ibamr
```

- “There is now a new ibamr install on dogwood (as of this morning). I will update all of the examples for that install over the next month. WARNING: I have not tested this yet!”
- “I recommend installing your own updated version of ibamr as well”; the old examples were to be kept in a folder `ibamr-examples-2018-03` (none survives in the course’s files). The reasons given for moving: two-boundary IBFE handling had changed, and the constraint-IB and sharp-interface methods were only in the new install.
- “Any example without a date appended on the end is for the 2016-11 version of IBAMR to run on killdevil. These examples will NOT run on dogwood.” The updated ones were “appended with 2018-03” (2018).
- The Makefile lines for both modules are in chapter X3 (dating code by its paths).

Today: no IBAMR module on Puma: your own autoibamr build (chapter 2).

“2-Run your IBAMR simulation on Dogwood” (2018), slides 11 and 20; “Get dogwood account and run a simulation” (2020); IBFE lecture 5, “Plans for June” (1 June 2020), slides 2–4.

SLURM ON DOGWOOD: SBATCH, SQUEUE, SCANCEL

- `sbatch script.sh` submits a job script; Slurm answers with the job's number.
- `squeue -u <onyen>` lists your jobs ("provide your onyen to just show your jobs"); the 2018 slide prints `squeue -u` with the name left out, and the 2020 guide writes `squeue -uonyen`, with the placeholder to replace.
- `scancel [JobID]` stops one.
- The commands are the same today; what changed is the partition names, the account line, and the way the program is launched inside the script.

> Today: `squeue --me`, `scancel <jobid>`, `sacct` for jobs that have ended (chapter 1).

"2-Run your IBAMR simulation on Dogwood" (2018), slides 4–5 (with Longleaf added in Lecture 8, 2019–20, slides 20–21); "Get dogwood account and run a simulation" (2020).

A FIRST SLURM SCRIPT, TWICE

Lecture 8 (2019–20), slides 22–23 (in the deck begun in April 2018) • historical, do not run

```
nano example.sh

#!/bin/bash
#SBATCH --job-name=first_slurm_job
#SBATCH --ntasks=1
#SBATCH --time=1:00
#SBATCH --mem=100
echo "Hello SLURM"

sbatch example.sh
```

2020 guide • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=first_slurm_job
#SBATCH --ntasks-per-node=16
#SBATCH --time=1:00:00
#SBATCH --mem=1000
echo "Hello SLURM"
```

- “each SBATCH switch below has two ‘-’ characters”.
- The lecture’s script asks for “1 core and 100MB of memory for 1 minute”, as its text says (`--time=1:00` is one minute, `--mem=100` is in MB).
- The 2020 guide changed the script (16 tasks per node, one hour, 1000 MB) but kept the old sentence: the text no longer matches.

Today: chapter 1’s first job script names the account and partition, asks for memory per core, and is checked with `squeue --me`.

“2-Run your IBAMR simulation on Dogwood” (2018), slides 6–7 = Lecture 8 (2019–20), slides 22–23; “Get dogwood account and run a simulation” (2020).

ONE-LINE JOBS: SBATCH --WRAP, AND SLURM'S DEFAULTS ON LONGLEAF

Lecture 8 (2019–20), slide 24 • historical, do not run

```
$ sbatch -t 1 -n 1 -o hello.out --wrap="echo hello SLURM"
```

The defaults the slide lists: partition `general` (`-p`), one task (`-n 1`), `--mem-per-cpu=4800` (MB), and a one-hour limit, written `-t 60`, `-t 1:00:00`, `-t 0-01:00` or `--time=1:00:00`; `-t 1-0` is one day.

- `--wrap` turns one command into a job without writing a script; `-o` names the output file.
- A slip in slide 26's example (a Python job on another group's path): "Time = 1 hour" for `-t 1`, which is one *minute* (a bare number is minutes), and "Number nodes = 1" for `-n 1`, which counts *tasks*.

Today: `--wrap` works on Puma too, once `--account` and `--partition` are added (chapter 1); a script is easier to keep and to repeat.

"2-Run your IBAMR simulation on Dogwood" (2018), slides 8 and 10 = Lecture 8 (2019–20), slides 24 and 26 (the slide 26 path is a third party's, not reproduced).

SHORTCUTS: SHOWQUEUE AND DELETEALLJOBS

Lecture 8 (2019–20), slide 25, as written • historical, do not run

```
alias showqueue='squeue -u YOUR_ONYEN'
alias deletealljobs='scancel -u YOUR_ONYEN’
```

“showqueue is basically the command: bjobs”;
“deletealljobs is basically the command: bkill 0”.

- Two aliases, typed at the prompt or kept in `~/.bashrc`, gave the old LSF habits new names.
- A slip: the second line closes with a curly quote (`’`), so the shell keeps waiting for the closing `'`.

Today: `alias showqueue='squeue -u $USER'`, or simply `squeue --me`; `scancel -u $USER` stops all your jobs (chapter 3).

“2-Run your IBAMR simulation on Dogwood” (2018), slide 9 = Lecture 8, “COMSOL workflow, export and run remotely” (2019–20), slide 25.

THE IBAMR JOB SCRIPT ON DOGWOOD (1/2): THE 2020 SCRIPTS

2020 guide, the first IBAMR job • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=first_ibamr_job
#SBATCH --ntasks-per-node=16
#SBATCH --time=2:00:00
#SBATCH --mem=1000
#SBATCH --partition=debug_queue
#SBATCH --output IBFE2D.out
mpirun ./main2d input2d
```

the course's 2D examples (`examples/legacy`), e.g. the rubber band • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=ibamr-rubberband
#SBATCH --ntasks-per-node=16
#SBATCH --time=2:00:00
#SBATCH --partition=debug_queue
#SBATCH --output ibamr2d.out

mpirun ./main2d input2d
```

- 16 tasks on one node, two hours, the `debug_queue` partition; `mpirun` with no count takes every task Slurm gave the job.
- "there are a number of partitions to which you can submit. Some have minimum numbers of ntasks and run times" (UNC's partition table).
- The other 2D course scripts follow it, with their own job names (`ibamr-pulsatile`, `ibamr-leaf`, `ibamr-cylinders`, `ibamr-spring`, `ibamr-2plates`); the 3D glycocalyx asks for the same 16 tasks and runs `main3d`.

Today: chapter 3's `job.slurm`: `--account`, `--partition=standard`, `--ntasks=4`, `--mem-per-cpu`, the modules, `enable.sh`, `mpirun -np "$SLURM_NTASKS"`.

"Get dogwood account and run a simulation" (2020); the job scripts of `examples/legacy/UNC-Examples` (2020).

THE IBAMR JOB SCRIPT ON DOGWOOD (2/2): THE FIRST ONE, APRIL 2018

"Run your IBAMR simulation on Dogwood" (2018), slide 12, "Sample 2D script"

• historical, do not run

```
#!/bin/bash
#SBATCH --job-name=first_ibfe_job
#SBATCH --ntasks=64
#SBATCH --time=2:00:00
#SBATCH --mem=1000
#SBATCH --partition=528_queue
#SBATCH --output IBFE2D.out

mpirun ./main2d input2d
```

- 64 tasks in the `528_queue` partition for a 2D example: Dogwood's large-job partitions had a minimum size, and smaller jobs waited with the reason `QOSMinCpuNotSatisfied` (slide 72).
- `--mem=1000` is 1000 MB per node in total, for all the tasks on it; the 2019 tips raise it to `--mem=50000`.
- The 2020 scripts (last slide) moved to `debug_queue` with 16 tasks per node.

Today: `--mem-per-cpu`, and as many tasks as the run needs (chapter 3).

The instructor's "2-Run your IBAMR simulation on Dogwood" (created April 2018), slide 12.

SCRIPTS THAT FORGOT `MPIRUN`

the course's IBFE examples 1-, 2- and 4- (2020) • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=first_ibfe_job
#SBATCH --ntasks=16
#SBATCH --time=2:00:00
#SBATCH --partition=debug_queue
#SBATCH --output IBFE2D.out

./main2d input2d
```

The script reserves 16 tasks but starts the program directly, not through `mpirun`: one process runs, and 15 reserved cores sit idle for the whole job. Nothing fails, so nothing warns you: the run takes as long as a one-core run, with 15 cores reserved and unused.

The IBFE examples 7- and 9- (built with `ibamr/0.6.0`) have `mpirun`.

Today: `mpirun -np "$SLURM_NTASKS" ./build/main2d input2d` (chapter 3).

The job scripts of `examples/legacy/UNC-Examples` (1-, 2-, 4-, 7- and 9-series IBFE folders, 2020).

BIG 3D JOBS: 528_QUEUE, AND RESTARTS

the course's 3D IBFE examples (2020) • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=528_queue
#SBATCH --ntasks-per-node=32
#SBATCH -N 2
#SBATCH --time=72:00:00
#SBATCH --partition=528_queue
#SBATCH --output IBFE3D.out

mpirun ./main3d input3d
```

G. Hobson's tutorial (2020), slide 30, "Restarting a job" • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=re48
#SBATCH --ntasks-per-node=44
#SBATCH -N 4
#SBATCH --time=72:00:00
#SBATCH --partition=528_queue
#SBATCH --output output

mpirun ./main3d input3d $PWD/restart_IB3d 995000
```

- Dogwood's `528_queue` partition took the big parallel jobs: here 2 nodes of 32 tasks (64 cores) for up to 72 hours. The job name repeats the partition's name, harmlessly.
- A run longer than the time limit continues from a restart file: the restart folder and the step number follow the input file on the command line (here step 995000, on 4 nodes of 44 tasks).
- A restart must use the same number of processes as the run that wrote it.

Today: restarts work the same way on Puma (chapter 3); a long run is split into jobs within the partition's time limit.

The 3D job scripts of `examples/legacy/UNC-Examples` (horseshoe crab, flow tank, two wings; 2020); Gabrielle Hobson, "Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster" (23 September 2020), slide 30.

WATCHING A DOGWOOD JOB

- Keep a record of each job: its number, what it was, its parameters (the 2014 advice, kept).
- `squeue -uonyen` (with your ONYEN) while it waits and runs.
- The run's log, `IB2d.log`, read with `nano` or `vi` as in 2014 (slides 34–35); the guide keeps the 2014 slip about Esc in vi.
- "Please see lecture 1 for how to visualize this data!"

> Today: `squeue --me`; `tail -f` on the log; `sacct -j <jobid>` after the job; the job's `.out` file for errors (chapters 1 and 3).

"Get dogwood account and run a simulation" (2020), the sections on watching and copying a job.

WHEN A DOGWOOD JOB WAITS, STOPS OR CRAWLS

“The number of nodes requested can freeze your job”: “I submitted a bunch of jobs on Dogwood, but they are all pending with the code (QOSMinCpuNotSatisfied).” “Solution: You need to make sure you are above the minimum number of nodes you must request.”

“My job randomly stopped” • historical, do not run

```
Look at the end of both the IB2d.log / IB3d.log files to see what
      ↪ happened.
Also look at the .out file to see if what happened. This file is
      ↪ named after whatever you put in your script.
PETSC errors happen if you did not dedicate enough memory.
In your script, increase the mem option:
#SBATCH --mem=50000
You may not have devoted enough time in your job.
In your script, increase the time option
#SBATCH --time=4:00:00
```

“My job is super slow” • historical, do not run

```
IBAMR can trip up if you use the wrong number of nodes (too many
      ↪ nodes, or some weird number).
Try to make the number of nodes a power of 2.
2D simulations can really slow down if you use too many nodes.
Make sure your DT isn't too small in the input file.
```

- “nodes” here means MPI processes (tasks); the pending reason is about the number of CPUs a partition requires.
- A PETSc error is one possible sign of too little memory, not the only cause; `--mem=50000` is 50,000 MB (about 49 GB) per node.
- “A power of 2” is a habit, not a rule; too many processes for a small 2D run does slow it down.

Today: chapter 3’s table of first errors and chapter 16’s list of slow and failing runs are these tips, rewritten for Puma.

The instructor’s “2-Run your IBAMR simulation on Dogwood” (2018–20), slides 13–16, and “Debugging Tips” (July 2019), the same text.

LONGLEAF: A COMSOL BATCH RUN (2019)

Lecture 8 (2019–20), slide 17, the log condensed • historical, do not run

```
module load comsol/5.4
# the COMSOL batch log (Tue Feb 19 2019):
#   COMSOL 5.3a (Build: 348) opening
#   /nas/longleaf/home/<onyen>/COMSOL/Barnacle.mph
File created with a later version of COMSOL Multiphysics. - Version:
      ↪ COMSOL 5.4.0.225
```

- The lecture ran a COMSOL model (heat in a barnacle) as a batch job on Longleaf, the single-node cluster.
- “NOTE: module load comsol/5.4”: the log shows COMSOL 5.3a refusing a file saved with 5.4, so the job had run the wrong version. Load the version the file was saved with.
- COMSOL itself is outside this series; the lesson carries over to any program with modules: name the version.

Lecture 8, “COMSOL workflow, export and run remotely” (2019–20), slide 17 (the user ID replaced).

VISIT ON DOGWOOD AND LONGLEAF (2020) (1/2): INSTALLING IT YOURSELF

G. Hobson's tutorial (2020), slides 4–5 • historical, do not run

```
cd ~
mkdir visit
cd visit
scp visit3_1_2.linux-x86_64-rhel7-wmesa.tar.gz <onyen>@dogwood.unc.
      ↪ edu:~/visit/
 tar -xvf visit3_1_2.linux-x86_64-rhel7-wmesa.tar.gz
/nas/longleaf/home/<onyen>/visit/visit3_1_2.linux-x86_64/bin/visit -
      ↪ cli -nowin -s script.py
```

The download: VisIt 3.1.2, "the last option" on the executables page, a Red Hat Enterprise Linux 7.5 build that "Includes Mesa support for rendering without a display".

- Download on your computer, `scp` to the cluster, unpack, and run by its full path: no module and no build.
- On the slides the command is written with en dashes (`–cli –nowin –s`, in red), which VisIt does not read as options; the full-path line is right.
- A message at start-up, that the ADIOS2 database plugin could not load a library, is harmless: that one reader is missing, the others work.

Today: the same recipe for Puma, with a current VisIt for Rocky Linux 9 (chapter 31).

Gabrielle Hobson, "Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster" (Scripps Institution of Oceanography, UCSD; 23 September 2020), slides 3–5 and 15–16 (the user ID replaced).

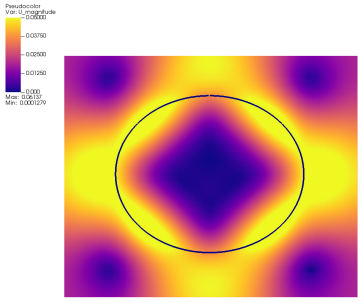
VISIT ON DOGWOOD AND LONGLEAF (2/2): PATHS IN THE SCRIPTS

G. Hobson's tutorial (2020), slide 10 (paths with placeholders; the correlation lines left out)

• historical, do not run

```
string_dumps = "localhost:/nas/longleaf/home/<onyen>/code/rubber_band/
      ↪ viz_IB2d/dumps.visit"
string_mesh = "localhost:/nas/longleaf/home/<onyen>/code/rubber_band/
      ↪ viz_IB2d/lag_data.visit"
string_output = "/21dayscratch/scr/<o>/<n>/<onyen>/flow_fields/
      ↪ rubber_band"
OpenDatabase(string_dumps)
OpenDatabase(string_mesh)
```

- A script recorded on a laptop or desktop has that computer's paths; on the cluster each must become `localhost:` plus the cluster's path. Slide 9 of the tutorial has the slips (a missing colon after `localhost`, a curly quote, a missing opening quote); slide 10 is right.
- "commands to do with 'spontaneous states', like when you click the zoom icon in the GUI, do not work on the cluster".
- VisIt 3.1 on the cluster "has trouble reading .VTK databases": the tutorial copies a `dumps.visit` into the VTK folder and lists the `.vtk` files in it.



A frame of the rubber band rendered on the cluster with no display (the tutorial's figure).

Today: chapter 31 (paths, batch rendering, VTK series opened directly in VisIt 3.5).

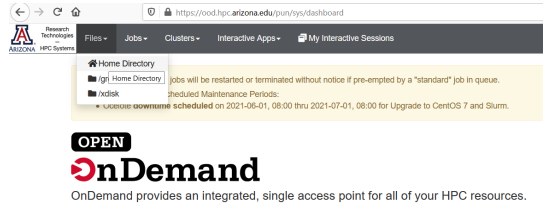
Gabrielle Hobson, "Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster" (2020), slides 9–10, 13, 17 (her figure), 20 and 24.

DOGWOOD AND LONGLEAF, THEN AND NOW

	Dogwood, Longleaf (2018–2020)	Puma (2025–26; chapters 1–3)
account	ONYEN Services, or an e-mail to Research Computing	HPC portal, sponsored by a PI group
log in	`ssh <onyen>@dogwood.unc.edu`; VPN and Duo off campus	`ssh <netid>@hpc.arizona.edu`, Duo, then `shell`
run folder	`/21dayscratch/scr/o/n/<onyen>` (21 days)	`/xdisk/<pi>/<netid>` (150 days, +150)
keep	`~/ms`, up to 1 TB	`/groups/<pi>`, 500 GB; your own drive
software	`module add ibamr/2018-03` or `ibamr/0.6.0`	autoibamr; `module load gnu13 openmpi5`; `enable.sh`
job script	`--ntasks-per-node=16`, `--partition=debug_queue` or `528_queue`; `mpirun ./main2d input2d`	`--account`, `--partition=standard`, `--ntasks=4`; `mpirun -np "$SLURM_NTASKS"`
watch, stop	`squeue -u <onyen>`, `scancel`	`squeue --me`, `scancel`, `sacct`
VisIt	your own 3.1.2 (RHEL 7, Mesa) by its full path	module 3.4.2 or your own 3.5.0

UARIZONA'S CLUSTERS, 2021–2024: PUMA, OCELOTE, EL GATO

- Three clusters behind one bastion host: Puma (the newest), Ocelote and El Gato. "After you login to the bastion host, select puma or ocelot"; "Once connected, you are using a login node", and "software is not available on the login nodes".
- Puma and Ocelote used Slurm ("Uses SLURM scheduling software", 2022), and by 2023 El Gato was a third Slurm target. Ocelote moved to Slurm in June–July 2021 (the banner on the right; the July 2021 slides expected it "online July 12"); no source in the course's files uses Ocelote's earlier scheduler.
- The notes call Ocelote "ocelot" throughout; the shortcut at the bastion was `ocelote`.
- El Gato was shut down on 29 July 2026 and Ocelote on 17 August 2026; since then Puma is the only UA cluster (chapter 1).



Open OnDemand at Arizona, spring 2021: windfall jobs "will be restarted or terminated without notice if pre-empted by a 'standard' job"; "Ocelote downtime scheduled on 2021-06-01, 08:00 thru 2021-07-01, 08:00 for Upgrade to CentOS 7 and Slurm."

"HPC and computing resources @ UA" (about 2022), slides 21 and 29, and the screenshot of the OnDemand dashboard on its slide 24 (spring 2021); "HPC @ UA, running ib2d and IBAMR" (2021–22); chapter 1 for the 2026 shutdowns.

ACCOUNTS AND GROUPS (2021–2024)

- 2021–22: request HPC on the university's account page ("Manage your Accounts", High Performance Computing); "I will need to add you to my group".
- "Note that there is a charge for simulations – please use the windfall partition unless you have permission for standard." And in OnDemand: "if you select Standard, your PI will be charged for the time. Although windfall is not recommended, it is free and can be used for low risk tasks."
- 2024: the account at the HPC portal (`portal.hpc.arizona.edu`) with a sponsorship request to the PI.

Today: the portal and a PI sponsor as in 2024; `standard` jobs use the PI group's allocation of CPU hours (`--account=<your PI group>`), `windfall` is free and can be pre-empted (chapter 1).

"HPC @ UA, running ib2d and IBAMR" (2021–22), slides 3–4; "HPC and computing resources @ UA" (about 2022), slides 17 and 27; UA Lecture 1 (July 2024).

CHOOSING A CLUSTER AT THE BASTION

UA notes, 2021–2024 (<netid> for the user ID) • historical, do not run

```
ssh <netid>@hpc.arizona.edu
# two-step login, then at the bastion host:
puma        # or ocelote, or elgato
```

“you are currently logged into the Bastion Host. You must now log into puma, ocelot, or elgato. on the cluster. Let’s log into puma.” (2024)

- The bastion host (`hpc.arizona.edu`) is only a gateway: a cluster’s name typed there opened a login node, and by 2023 the same names were shortcuts that “change the target cluster” for jobs.
- The 2021–22 slides say “Type puma/ocelot to work on this cluster”.

Today: `ssh <netid>@hpc.arizona.edu`, then `shell` (Puma is the only cluster; chapter 1).

UA Lecture 1 (July 2024), the login section (its screenshot of the shortcut list retyped); “HPC @ UA, running ib2d and IBAMR” (2021–22), slides 10 and 16.

VPN AND TWO-STEP LOGIN, 2021–2024

- 2021–22: the Cisco VPN client to `vpn.arizona.edu` from off campus, with a second password of `push` or `phone` for Duo.
- 2024: “Please install the cisco client”.
- FileZilla and `sftp` asked for the NetID password and then `1` for a Duo push (or a passcode).

> Today: no VPN for SSH or Open OnDemand; Duo at the bastion host; the HPC VPN (`vpn.hpc.arizona.edu`) only for programs that cannot pass the bastion (chapter 1).

“HPC @ UA, running ib2d and IBAMR” (2021–22), slides 6–8; “HPC and computing resources @ UA” (about 2022), slides 4 and 19; UA Lecture 1 (2024).

MOVING FILES, 2021–2024: FILEZILLA, SFTP, ONDEMAND

"HPC @ UA" (2021–22), slides 9–10 • historical, do not run

```
# from the folder on your computer:
sftp <netid>@filexfer.hpc.arizona.edu
# "Once logged in, type puma or ocelot to use one of these
      ↪ servers"
put file
put -r folder
get file
get -r folder
```

FileZilla: protocol SFTP, host `filexfer.hpc.arizona.edu`, logon type Interactive, user your NetID; in the Transfer Settings, a limit of 1 connection (one Duo approval per connection).

- Open OnDemand's file browser (`ood.hpc.arizona.edu`): drag and drop.
- Slips: a slide on the university's general server writes `sftp netid@uarizona.edu` for `u.arizona.edu` (as the same slide says three lines earlier); "a terminal that allows 2-step authentication (Cygwin for PC, terminal for Mac)" : Cygwin was not needed; Windows' own `ssh` (PowerShell, Windows Terminal), WSL or PuTTY do this (chapter 1).
- The `sftp` slide's "type puma or ocelot" does not apply to `filexfer`, which is the file-transfer host itself.

Today: the same FileZilla settings and `sftp`, `scp` or `rsync` to `filexfer.hpc.arizona.edu` (chapter 1).

"HPC @ UA, running ib2d and IBAMR" (2021–22), slides 7–12; "HPC and computing resources @ UA" (about 2022), slides 10 and 20–25; UA Lecture 1 (2024).

EL GATO AND PUMA IN OPEN ONDEMAND (2021–22)

- “You can use Jupyter notebooks, MATLAB, and other software through a web interface using Puma and ElGato” (at `ood.hpc.arizona.edu`, also written `OnDemand.hpc.arizona.edu`).
- “Go to interactive apps and select ElGato Jupyter Notebook”; the form asked for the hours, the cores, the PI group and the queue, with the advice on windfall and standard of slide 78.
- The spring-2021 OnDemand menu (its screenshot carries the Ocelote banner of slide 77) listed the El Gato apps beside Puma’s.

Today: `ood.hpc.arizona.edu`, with Puma’s interactive apps (a desktop, Jupyter, MATLAB); VisIt runs on the desktop (chapters 1 and 31).

“HPC @ UA, running ib2d and IBAMR” (2021–22), slide 5; “HPC and computing resources @ UA” (about 2022), slides 18 and 26–27 (screenshots not reproduced: they show a session card and a group name).

INTERACTIVE, AND THE EL GATO HINT (2022–24)

UA documentation example in the 2022 deck, slide 31 • historical, do not run

```
[<netid>@wentletrap ~]$ interactive
srun: job 57384 queued and waiting for resources
srun: job 57384 has been allocated resources
[<netid>@r1u03n2 ~]$
```

the 2022 deck, slide 32 • historical, do not run

```
module avail
module load <software>
module list
module unload <software>
```

- `interactive` asked Slurm for a compute node and opened a shell there (the prompt changes from the login node to the node); 2024: “to request a single core interactive session for one hour”.
- In 2024 it ran in the `windfall` queue by default, and its message suggested the `elgato` command.
- Software (modules) is loaded on the compute node, not on the login node.

Today: `interactive -a <your group>` (chapter 1); El Gato is gone.

“HPC and computing resources @ UA” (about 2022), slides 31–32 (copied from the UA documentation; its `netid` written `<netid>`); UA Lecture 1 (July 2024) and its screenshot of `interactive`, retyped as a summary.

STORAGE, 2021–2024: /HOME, /XDISK, /GROUPS, /CONTRIB

- “IB data is large, and I have created a new temporary space. The path to this folder is /xdisk/<group>”; run and write the visualization data there, and download it promptly.
- “Permanent space at /groups/<group> – 500 GB, currently almost full.”
- `/contrib/<group>`: where the 2021 IBAMR install lived (slide 89).
- A slip (2024): “each user has a folder . . . the xdisk is deleted every 300 days”. An `/xdisk` allocation lasts 150 days and can be extended once by 150.
- The deck’s clean-up line, `rm –r my_ib_sim_folder`, has an en dash (in red).

Today: `/home` (50 GB), `/xdisk/<pi>/<netid>` (150 days, extendable once), `/groups/<pi>` (500 GB); IBAMR in your own home through autoibamr (chapters 1–2).

IB2D ON PUMA AND OCELOTE (2021–22)

- Upload IB2d (unzipped, with its old `viz_IB2d` and `hier_IB2d_data` folders removed) through OnDemand or `sftp ... put -r`; or, the "Better Option", `git clone https://github.com/nickabattista/IB2d.git` on the cluster and `git pull origin master` to update.
- Run an example from its own folder, outside the IB2d tree, with an absolute path to IB2d's solver in `main2d.m`: `addpath('/home/uXX/<netid>/IB2d/matIB2d/IBM_Blackbox')` in place of the relative `addpath`.
- Afterwards: download `viz_IB2d` (web or `get -r viz_IB2d`), then `rm -r viz_IB2d`.

> Today: clone IB2d on Puma; run its shipped examples in place, keep your own copies outside the clone, as in 2021 (chapter 1; Part II), and submit MATLAB as a batch job (chapter 1).

Two course copies of IB2d's `main2d.m` still hard-code such a path (rubber band, sawtooth, in `originals/Old files/AZ-Examples`); `uXX` stands for a numbered folder of the home path, `<netid>` for the user.

"HPC @ UA, running ib2d and IBAMR" (2021–22), slides 13–15 and 20.

THE MATLAB BATCH SCRIPT FOR IB2D (2021–22)

"HPC @ UA" (2021–22), slide 19, `myscript.sh` • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=Lauras_job
#SBATCH --ntasks=1
#SBATCH --mem=16gb
#SBATCH --time=04:00:00
 #SBATCH --partition=windfall

module load matlab
matlab -nodisplay -nosplash < main2d.m
```

slide 16 • historical, do not run

```
ssh <netid>@hpc.arizona.edu
# "Type puma/ocelot"
cd <example folder>
sbatch myscript.sh
squeue -u<netid>
```

- One core, 16 GB, four hours, MATLAB reading `main2d.m` from standard input.
- A slip: the partition line starts with a space. Slurm reads `#SBATCH` lines only from the first column, so it ignores this one: the job does not go to `windfall` but to the default partition, or is refused, since there is no `--account` either.
- The AZ copy of the rubber band's `script.sh` (2021) is the same with the job name `ib2d` and no leading space.

Today: chapter 1's MATLAB script: `--account`, `--partition=standard`, `module load matlab`, the same `matlab -nodisplay -nosplash < main2d.m`.

"HPC @ UA, running ib2d and IBAMR" (2021–22), slides 16 and 19; the AZ rubber band's `script.sh` (`originals/Old files/AZ-Examples`).

THE UARIZONA SLURM TEMPLATE, 2021–2022

"HPC @ UA" (2021–22), slides 17–18 (the UA Puma Quick Start template) • historical, do not run

```
#!/bin/bash
### PART 1: Requests resources to run your job.
### Optional. Set the job name
#SBATCH --job-name=hello_world
### Optional. Set the output filename.
### SLURM reads %x as the job name and %j as the job ID
#SBATCH --output=%x-%j.out
### REQUIRED. Specify the PI group for this job
#SBATCH --account=<PI GROUP>
### Optional. Request email when job begins and ends
### SBATCH --mail-type=ALL
### Optional. Specify email address to use for notification
### SBATCH --mail-user=<YOUR NETID>@email.arizona.edu
```

continued • historical, do not run

```
### REQUIRED. Set the partition for your job.
#SBATCH --partition=standard
### REQUIRED. Set the number of cores that will be used for
      ↪ this job.
#SBATCH --ntasks=1
### REQUIRED. Set the number of nodes
#SBATCH --nodes=1
### REQUIRED. Set the memory required for this job.
#SBATCH --mem=1gb
### REQUIRED. Specify the time required for this job, hhh:mm:ss
#SBATCH --time=00:01:00
### PART 2: Executes bash commands to run your job
module load gnu/5.4.0
cd ~/hello_world
./hello_world
sleep 10
```

The university's template, with each line explained (the dashed separator lines and three comment lines of the original are left out here). `### SBATCH` (three hashes and a space) switches a line off: the e-mail lines are there to be turned on. `%x-%j.out` names the output file after the job and its number; `module load gnu/5.4.0` loads the compilers of the time. Today: chapter 1 builds its first script from the same template, with `--mem-per-cpu=4gb` and `module load gnu13`; IBAMR jobs load `gnu13 openmpi5` (chapter 3).

"HPC @ UA, running ib2d and IBAMR" (2021–22), slides 17–18, quoting the UA Puma Quick Start of the time (separator lines and three comments omitted); "HPC and computing resources @ UA" (about 2022), slides 33–35.

IBAMR IN 2021: OCELOTE FOR IBFE, SRUN ON PUMA

"HPC @ UA", versions of July 2021 to January 2022, slide "Running IBAMR" • historical, do not run

```
Mpiruns using IBFE can only be done on ocelot
This cluster should be online July 12.
We can run simulations using IBFE by using srun on puma (2D runs).
      ↪
If you are  not using finite elements / libmesh (e.g. fiber based
      ↪  IBM), then you can use mpirun on puma.
```

`puma_script.sh` (2021) • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=IBAMR-test
#SBATCH --ntasks-per-node=1
#SBATCH --nodes=1
#SBATCH --time=04:00:00
#SBATCH --partition=windfall
#SBATCG --output IBAMR2D.out

srun ./main2d input2d
```

`ocelot_script.sh` (2021) • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=IBAMR-test
#SBATCH --ntasks-per-node=16
#SBATCH --nodes=1
#SBATCH --time=04:00:00
#SBATCH --partition=windfall
#SBATCG --output IBAMR2D.out

mpirun ./main2d input2d
```

- In July 2021 the group's IBAMR could run IBFE in parallel only on Ocelote; on Puma, IBFE ran with `srun` on one task. The text was replaced in February 2022, when the group's newer install worked (next slide).
- `#SBATCG`: a typo, so Slurm ignores the line and writes `slurm-<jobid>.out`.

"HPC @ UA, running ib2d and IBAMR", versions of 11 July 2021 to 20 January 2022 (git history); the AZ triangle's `ocelot_script.sh` and `puma_script.sh` (6 July 2021).

GROUP IBAMR INSTALLS, 2021–2025

"HPC @ UA" (2022), slide 23, the Makefile paths • historical, do not run

```
IBAMR_SRC_DIR = /contrib/<group>/ibamr/0.7.1/include/IBAMR
IBAMR_BUILD_DIR = /contrib/<group>/ibamr/IBAMR-0.7.1
LIBSNEW = $(LIBS) -lnetcdf -lcurl
*These directories are for the old IBAMR install where IBFE does not
      ↪ work. ...
The new installation where IBFE works is as follows:
IBAMR_SRC_DIR = /groups/<group>/ib10/ibamr/IBAMR
IBAMR_BUILD_DIR = /groups/<group>/ib10/ibamr/ibamr-objs-opt
LIBSNEW = $(LIBS) -lnetcdf -lcurl
```

slide 24, the change to each IBFE main file • historical, do not run

```
//AutoPtr<ExodusII_IO> exodus_io(uses_exodus ? new ExodusII_IO(mesh) :
      ↪  NULL);
std::unique_ptr<ExodusII_IO> exodus_io(uses_exodus ? new ExodusII_IO(
      ↪ mesh) : NULL);
```

- 2021: IBAMR 0.7.1 in `/contrib/<group>`; already then the IBFE mains needed `std::unique_ptr` for libMesh (chapter X3). In 2022 the deck called it "Old version, IBFE does not work".
- 2022: the group's `ib10` build in `/groups/<group>`, "where IBFE works" (presumably 0.10, by its name).
- July 2025: an `ib16` build, `/groups/<group>/ib16/Code/IBAMR` and `.../ibamr-objs-opt`, with a Makefile that builds IBAMR's own `example.cpp` programs (Lecture 6, 2025).
- The 2024 "Lecture 0" repeats slides 23–24 for the course's UNC examples.

Today: each user's own autoibamr install and CMake (chapters 2–3). How these group installs were built is not in the course's files (chapter X2 lists them).

"HPC @ UA, running ib2d and IBAMR" (version of 23 June 2022), slides 21–24; "Lecture 0 – Updating UNC Examples" (July 2024); Lecture 6 (updated July 2025), slides 3–4. <group> in place of the group's name.

IBAMR JOB SCRIPTS AT UARIZONA (1/2): 2021–2024

"HPC @ UA" (2021–22), slide 25 (slide 21 in July 2021) • historical, do not run

```
#!bin/bash

#SBATCH --job-name=IBAMR-test
#SBATCH --ntasks-per-node=16
#SBATCH --nodes=1
#SBATCH --time=04:00:00
#SBATCH --partition=windfall
#SBATCH --output IBAMR2D.out

mpirun ./main2d input2d
```

2021–22: 16 tasks on one node in `windfall`, with no account line; the first line `#!bin/bash` lacks the slash of `#!/bin/bash`, so the job looks for `bin/bash` in its own folder and fails at start.

UA Lecture 1 (2024) • historical, do not run

```
nano script.sh

#!/bin/bash
#SBATCH --job-name=IBAMR-test
#SBATCH --ntasks=1
#SBATCH --nodes=1
#SBATCH --time=04:00:00
#SBATCH --partition=standard
#SBATCH --account=<group>
#SBATCG --output IBAMR2D.out
mpirun ./main2d input2d

sbatch script.sh
squeue -unetid
```

2024: `standard` with `--account=<group>`, but one task, so `mpirun` starts a single process (fine for a first test, slow for a real run); and the `#SBATCG` typo again. The 2024 course examples' `script.sh` (rubber band, channel, beam, cylinders) is this script with the typo fixed; the glycocalyx runs `main3d input3d`. Today: chapter 3's `job.slurm`.

"HPC @ UA, running ib2d and IBAMR" (2022), slide 25; UA Lecture 1 (July 2024); the job scripts of `examples/legacy/UA-Examples` (2024). `<group>` in place of the group's name.

IBAMR JOB SCRIPTS AT UARIZONA (2/2): IBFE, FIBRE STRESS, 3D

2024 UA IBFE examples 1- and 2-, `script.sh` • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=IBAMR-test
#SBATCH --ntasks-per-node=1
#SBATCH --nodes=1
#SBATCH --time=04:00:00
#SBATCH --partition=standard
#SBATCH --output=IBAMR.out
#SBATCH --account=<group>

srun ./main2d input2d
```

3D horseshoe crab and flow tank (UA copies) • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=528_queue
#SBATCH --ntasks-per-node=32
#SBATCH -N 2
#SBATCH --time=72:00:00
#SBATCH --partition=standard
#SBATCH --output IBFE.out
#SBATCH --account=<group>

mpirun ./main3d input3d
```

fibre-stress examples (2022) • historical, do not run

```
#!/bin/bash
#SBATCH --job-name=AT3
#SBATCH --ntasks-per-node=94
#SBATCH -N 1
#SBATCH --time=12:00:00
#SBATCH --partition=standard
#SBATCH --account=<group>
#SBATCH --output output1

srun ./main2d input2d
```

- IBFE with `srun` on one task, as in 2021; a full 94-core Puma node for 12 hours for a 2D fibre-stress run (`AT3` for the moving-stress examples, `AT2` for the static one); the Dogwood partition's name kept as a job name in 3D.
- The maintained versions run each fibre mode on 4 cores in under a minute (IBFE/06).

The job scripts of `examples/legacy/UA-Examples` (2022–24); `examples/IBFE/06-active-tension` README. `<group>` in place of the group's name.

ARIZONA 2021–2024, THEN AND NOW

	Puma, Ocelote, El Gato (2021–2024)	Puma (2025–26; chapters 1–3)
log in	VPN off campus; `ssh <netid>@hpc.arizona.edu`; `puma`, `ocelote` or `elgato`	no VPN; `ssh <netid>@hpc.arizona.edu`; `shell`
web	`ood.hpc.arizona.edu` (also written `OnDemand.hpc...`), El Gato and Puma apps	the same address, Puma apps
files	FileZilla or `sftp` to `filexfer.hpc.arizona.edu`	the same
storage	`/xdisk/<group>`, `/groups/<group>`; “300 days” [150 + 150]	`/xdisk/<pi>/<netid>`, `/groups/<pi>`, `/home`
IBAMR	group installs: `/contrib` 0.7.1, `/groups` `ib10`, `ib16`; Makefiles	your own autoibamr (0.16.0 on Puma); CMake
job script	`windfall` without account (2021); one task; `srun` for IBFE; `#SBATCG`	`--account`, `standard`, 4 tasks, `mpirun -np "$SLURM_NTASKS"`
modules	`gnu/5.4.0`; `matlab`	`gnu13`, `openmpi5`; `matlab`; `visit`
interactive	`interactive` (windfall; “Consider executing the elgato command”)	`interactive -a <your group>`

A 2023 adaptation of G. Hobson’s VisIt tutorial for the UA cluster exists in the course’s files; its slides do not name its author, and it is not reproduced here. VisIt on Puma is chapter 31.

OLD COMMAND OR PATH, AND TODAY'S (1/11): ACCOUNTS AND LOGINS

old (era)	today on Puma	where
ONYEN Services, "subscribe to" Kure, KillDevil, Dogwood; e-mail to Research Computing (UNC, 2014–20)	an HPC portal account; a PI sponsors you	ch. 1
`ssh <onyen>@kure.unc.edu`, `@killdevil.unc.edu`, `@dogwood.unc.edu`	`ssh <netid>@hpc.arizona.edu`, then `shell`	ch. 1
`puma`, `ocelote`, `elgato` at the bastion (UA, 2021–24)	`shell` (Puma is the only cluster)	ch. 1
SSH Secure Shell and File Transfer; PuTTY; Cygwin	a terminal (Mac, Linux, Windows OpenSSH or WSL), PuTTY	ch. 1
UNC VPN and Duo; `vpn.arizona.edu` before logging in	no VPN for SSH or OnDemand; `vpn.hpc.arizona.edu` only for programs that cannot pass the bastion	ch. 1
`scp -r folder <onyen>@<host>:/netscr/<onyen>`	`scp -r folder <netid>@filexfer.hpc.arizona.edu:/xdisk/<pi>/<netid>/`, or `rsync -av`	ch. 1
lab servers: `useradd onyen`, `edquota username`, per-user disk quotas (THLinux, 2011–13)	an HPC portal account; group storage	ch. 1

The old commands as this chapter shows them, checked against chapter 1.

OLD COMMAND OR PATH, AND TODAY'S (2/11): FILES AND STORAGE

old (era)	today on Puma	where
FileZilla to Kure or Dogwood; `sftp <netid>@filexfer.hpc.arizona.edu`, then "type puma or ocelot" (UA, 2021–22)	FileZilla or `sftp` to `filexfer.hpc.arizona.edu` (Interactive, 1 connection); nothing to type after logging in	ch. 1
OnDemand (`ood.hpc.arizona.edu`), El Gato apps	the same address, Puma apps	ch. 1, 31
a home with a quota (10 GB for one account in 2014, "∼ 5 GB" in 2016), `quota`, `du -hs`; `/nas/longleaf/home/<onyen>`	`/home` (50 GB)	ch. 1
`/netscr/<onyen>`, `/21dayscratch/scr/o/n/<onyen>` (21 days); `/pine/scr/o/n/<onyen>` (Longleaf; period not stated)	`/xdisk/<pi>/<netid>` (150 days, extendable once by 150)	ch. 1
mass storage `~/ms` (1 TB); `bsub -q ms cp ...`	`/groups/<pi>` (500 GB) and your own drive	ch. 1
"xdisk is deleted every 300 days"	150 + 150 days at most	ch. 1

The old commands as this chapter shows them, checked against chapter 1.

OLD COMMAND OR PATH, AND TODAY'S (3/11): MODULES AND BUILDS

old (era)	today on Puma	where
`module initadd ibamr/3803` (with `openmpi_intel/11.1 ...`); `ibamr/2016-11`; `module add ibamr/2018-03`, `ibamr/0.6.0`	your own IBAMR from autoibamr; in each job `module load gnu13 openmpi5` and `source ~/autoibamr/configuration/enable.sh`	ch. 2, 3
`module add openmpi_gcc; module initadd openmpi_gcc`; `export FC=gfortran CC=gcc ...`	`module load gnu13 openmpi5` (on a compute node)	ch. 1
`export LD_LIBRARY_PATH=/nas02/apps/intel-11.1/...`	not needed	ch. 2
`module load gnu/5.4.0` (UA template)	`module load gnu13 openmpi5`	ch. 1
Makefiles with `IBAMR_SRC_DIR` and `IBAMR_BUILD_DIR` (`/nas02/apps/ibamr-3803/...`, Dogwood, `/contrib/<group>`, `/groups/<group>/ib10`, `ib16`) and `LIBSNEW`	a `CMakeLists.txt`; `cmake -B build`, `cmake --build build`	ch. 3; X3
`make clean; make main2d` on the login node	build in a batch job or an `interactive` session	ch. 2, 3
`AutoPtr<ExodusII_IO>` to `std::unique_ptr` (UA, 2021)	the full porting checklist	ch. 3; X3
`module initlist`, `module initrm`, `module remove openmpi___` (2014)	no lists kept across logins: load modules in each job script	ch. 1
`IBAMR_SRC_DIR = $HOME/sfw2/...` in a Makefile (2013)	a `CMakeLists.txt`	ch. 3

The old commands as this chapter shows them, checked against chapters 1–3.

OLD COMMAND OR PATH, AND TODAY'S (4/11): SCHEDULERS (1)

old (era)	today on Puma	where
lab server: `mpirun -np 4 ./main2d input2d` in `screen`; `ps -u`; `kill`	`sbatch job.slurm`; `squeue --me`; `scancel`	ch. 3
`bsub -q week -n 8 mpirun ./main2d input2d`	`#SBATCH --ntasks=8` … `mpirun -np "$SLURM_NTASKS" ./build/main2d input2d`; `sbatch job.slurm`	ch. 3
`bjobs`, `bjobs -uonyen`, `squeue -u<onyen>`, `squeue -unetid`	`squeue --me`	ch. 1
`bkill <id>`, `bkill 0`, `scancel -u YOUR_ONYEN`	`scancel <id>`, `scancel -u $USER`	ch. 3
queue `week` (LSF); partitions `debug_queue`, `528_queue`, `general`; `windfall` with no account (UA, 2021)	`standard` with `--account=<your PI group>`, `windfall`, `high_priority`	ch. 1
e-mail when a job ends (LSF's default)	`#SBATCH --mail-type=ALL` and `--mail-user`	ch. 1
`#SBATCH --time=1:00` (1 min), `-t 1` (1 min, not 1 h), `--mem=100` (MB)	`--time=hh:mm:ss` (chapter 1's template: `00:10:00`; chapter 3's job: `00:30:00`), `--mem-per-cpu=4gb`	ch. 1, 3
`bqueues`; queues `debug`, `int`, `day`, `week` (LSF)	partitions `standard`, `windfall`, `high_priority`; `interactive`; the reason column of `squeue --me`	ch. 1

The old commands as this chapter shows them, checked against chapters 1 and 3.

OLD COMMAND OR PATH, AND TODAY'S (5/11): SCHEDULERS (2)

old (era)	today on Puma	where
`bjobs -l`, `bhist`, `bpeek`	`squeue --me`; `sacct -j <id>`; `tail -f` on the log or the `.out` file	ch. 1, 3
`screen` … `./main2d input2d &` … Ctrl+A Ctrl+D (lab servers, 2013)	a batch job; never on the login node	ch. 3

The old commands as this chapter shows them, checked against chapters 1 and 3.

OLD COMMAND OR PATH, AND TODAY'S (6/11): JOB SCRIPTS

old (era)	today on Puma	where
`sbatch -t 1 -n 1 -o hello.out --wrap="..."`	valid with `--account` and `--partition`; a script is better	ch. 1
`srun ./main2d input2d` (IBFE, Puma 2021); `./main2d input2d` with no launcher (Dogwood IBFE)	`mpirun -np "$SLURM_NTASKS" ./build/main2d input2d`	ch. 3
`mpirun ./main3d input3d $PWD/restart_IB3d 995000` (`528_queue`)	the same restart arguments, the same number of processes	ch. 3
`interactive` (windfall by default; "Consider executing the elgato command")	`interactive -a <your group>`	ch. 1
`bmatlab main2d.m`; `bsub matlab -nodisplay -nosplash -singleCompThread -r ...`	a MATLAB batch script (`module load matlab`; `matlab -nodisplay -nosplash < main2d.m`)	ch. 1
`alias showqueue='squeue -u YOUR_ONYEN'`	`alias showqueue='squeue -u $USER'`	ch. 3
`bsub ... -M 10` (2017)	`#SBATCH --mem-per-cpu=4gb`	ch. 1
a bash loop of `bsub -q week -n 16 mpirun ./main2d input2d-files/input2d${i}` (2018)	a Slurm job array	ch. 3
`#SBATCH --mem=50000`, `--time=4:00:00`; pending with `QOSMinCpuNotSatisfied` (2019)	`--mem-per-cpu`, `--time`; one node, with `--ntasks` processes	ch. 3, 16

The old commands as this chapter shows them, checked against chapters 1, 3 and 16.

OLD COMMAND OR PATH, AND TODAY'S (7/11): WATCHING AND RESULTS

old (era)	today on Puma	where
`grep "dt = " IB2d.log`; `tail -f IB2d.log`	follow the job's log and `.out` file	ch. 3
nano (Ctrl+W, Ctrl+V); vi ("hit the esc key. You can now write")	nano; vi (`i` to type, Esc to stop)	ch. 1
"at least within a week", `tar -cvzf`, an external drive	the same advice; `rsync`; then `rm -r`	ch. 1, 3
a job that "finishes within a fraction of a second": "try resubmitting the job OR run it on a single node"	read the `.out` file and `sacct`; the error tables	ch. 1, 3
`module add visit` (2.5.2), `visit/2.12.3` (KillDevil)	`module load visit` (3.4.2) or your own 3.5.0	ch. 31
`./build_visit2_12_3 --mesa`, `make package`, `visit-install`	a VisIt release tarball and `visit-install` (Rocky 9)	ch. 31
VisIt 3.1.2 (RHEL 7, Mesa) by `scp`, `tar -xvf`, full path (Dogwood)	the same idea on Puma, with a current version	ch. 31
`./main2d input2d -stokes_ksp_rtol 1.0e-9` (also after `mpirun` or `bsub`)	still honoured; better in the input file's solver block	ch. 16

The old commands as this chapter shows them, checked against chapters 1, 3, 16, 28 and 31–32.

OLD COMMAND OR PATH, AND TODAY'S (8/11): VISIT AND IB2D

old (era)	today on Puma	where
`visit -cli -nowin -s script.py` on the login node (KillDevil, 2018) or by VisIt's full path (Dogwood, 2020)	by the full path or the `visit` module, in a batch job	ch. 31
`ssh -X`, then `visit` (KillDevil)	an OnDemand desktop, X11, or client–server	ch. 31
"cannot read .ex2 files" (KillDevil's VisIt 2.12.3); export to VTK	VisIt 3 reads ExodusII	ch. 28, 31
record scripts in VisIt 2.5 (Python 2)	VisIt 3.2 and later (Python 3)	ch. 31, 32
the `.VTK` work-around (a copied `dumps.visit`)	open the `.vtk` series directly	ch. 31
`module add git/1.8.5.3`, then `git clone` IB2d (Kure, 2016)	`git clone` on Puma, no module	ch. 1
an absolute `addpath('/home/uXX/<netid>/IB2d/...')` in `main2d.m`	the shipped examples run in place with IB2d's relative paths; your own copies outside the clone	ch. 1; Part II

The old commands as this chapter shows them, checked against chapters 1, 3, 28 and 31–32.

OLD COMMAND OR PATH, AND TODAY'S (9/11): SLIPS TO FIX WHEN YOU COPY (1)

as written	where	fix
`scp –r`, `mpirun –np`, `ps –u`, `rm –r`, `visit –cli –nowin –s` (en dashes, in red)	2012–2022 notes	one ASCII hyphen each: `-r`, `-np` ...
`Mpirun` (capital M)	2012 notes	`mpirun`: commands are case-sensitive
`...dogwood.unc.edu: /21dayscratch/...` (space after the colon)	2020 guide	no space after the colon
`#SBATCG --output ...`	UA scripts, 2021–24	`#SBATCH --output=...`
`#!bin/bash`	UA deck, 2021–22	`#!/bin/bash`
`#SBATCH --ntasks-per-nod=16`	a 2020 copy	`--ntasks-per-node=16`
`ssh onyen@killdevil.unc.edu` in the Dogwood guide	a 2020 copy	`dogwood.unc.edu`
`--ntasks=16` with `./main2d` (no `mpirun`)	UNC IBFE scripts, 2020	`mpirun -np "$SLURM_NTASKS" ...`

Each slip, checked in its source (October 2026).

OLD COMMAND OR PATH, AND TODAY'S (10/11): SLIPS TO FIX WHEN YOU COPY (2)

as written	where	fix
`-t 1` described as one hour; `-n` as nodes	2019–20 lecture	`-t 1` is one minute; `-n` counts tasks
`make cleandist`; `./main2d input2c`	2012 ReadMe	`make distclean`; `input2d`
"Dump.visit"	2012–18 decks	`dumps.visit`
curly quotes in code (”, ’)	2018–20 decks	straight quotes
vi: "hit the esc key. You can now write"	2014–2024 handouts	`i` to type, Esc to stop
nano: "Ctrl + w + v"	2014–2024 handouts	Ctrl+W, then Ctrl+V
`~/.bash-profile`, `authorized-keys`	2014 notes	`~/.bash_profile`, `~/.ssh/authorized_keys`
"VPN (very private network)"	2014–18 decks	virtual private network

Each slip, checked in its source (October 2026).

OLD COMMAND OR PATH, AND TODAY'S (11/11): SLIPS TO FIX WHEN YOU COPY (3)

as written	where	fix
`Sudo yum install ...`; "sudo-apt install"	THLinux slides, 2011–13	`sudo yum install`; `sudo apt-get install`
`& Ctrl+A Ctrl+D` (probably a prompt typo)	2013	press Ctrl+A, then Ctrl+D; `screen -r` to return
`$HOME` in a Makefile	2013	`$(HOME)`
`module remove openmpi___`	2014	the module's real name
`squeue -u` with no name	2018–20	`squeue -u <onyen>`; today `squeue --me`
`bsub -q week -n 32 -M 10 ./main3d input3d` (no `mpirun`)	2017	`bsub ... mpirun ./main3d input3d`
WO never read in the 2018 sweep; folders that disagree	2018	read WO as Freq is read; write and read the same folder (chapter 3: values by name, one folder per run)

Each slip, checked in its source (October 2026).

CREDITS, AND WHAT THIS CHAPTER DOES NOT COVER

Credits. The notes of this chapter were written by the instructor and, for the parts named on their slides, by Shannon Jones (2012), Dennis Evangelista (2014), Nick Battista (2014–16), Alex Hoover (2014), Christopher Strickland and Lindsay Waldrop (VisIt scripts, 2017–18) and Gabrielle Hobson (VisIt on the cluster, 2020). Notes whose author is not recorded stay unattributed, as do two copy scripts by a student of the 2018 course. All screenshots are from those notes: Shannon Jones's 2012 dialog, Nick Battista's 2014 `nano` and `vi` views, the 2014 Kure deck's VisIt window, Gabrielle Hobson's 2020 frame, and the UA OnDemand page of 2021.

Not covered here.

- PBS (`qsub`): no course note uses it.
- Retirement dates of the UNC clusters: not stated in the course's files.
- Building IBAMR on the lab's and UNC's machines: chapter X2 (the Arizona group installs' recipes are not in the course's files); the code changes between installs: chapter X3.
- COMSOL: outside this series (chapter 1b names it). The VisIt recipes themselves and IB2d: Parts V and II.
- Personal details: user names, home paths and group names are replaced by placeholders (`<onyen>`, `<netid>`, `<user>`, `<group>`, `<o>/<n>`, `uXX`, `<run_folder>`); the lab servers' host names, passwords and phone numbers are left out.

SUMMARY

- 2012–13: the lab's own servers, runs started by hand in `screen`, and rules for sharing the cores.
- 2012–18: UNC's Kure and KillDevil with LSF (`bsub`, `bjobs`, `bkill`), IBAMR modules from 2014, and the 21-day `/netscr`.
- 2018–20: Dogwood and Longleaf with Slurm, modules `ibamr/2018-03` and `0.6.0`, `/21dayscratch` and mass storage.
- 2021–24: Arizona's Puma, Ocelote and El Gato behind a bastion host, group installs of IBAMR, `/xdisk` and `/groups`.
- Each change of machine changed the same few lines: the host, the IBAMR module or install, the scheduler commands and the scratch folder.

> Reading an old note: find its machine and year (slides 5, 45 and 46), look the command up in slides 93–103, fix the slips, and use chapters 1–3 for today's way.